

NPSVC++: A Representation Learning Framework for Nonparallel Classifiers

Junhong Zhang^{ID}, Zhihui Lai^{ID}, *Member, IEEE*, Jie Zhou^{ID}, *Member, IEEE*, and Guangfei Liang^{ID}

Abstract—This article focuses on a specific family of classifiers called nonparallel support vector classifiers (NPSVCs). Different from typical classifiers, the training of an NPSVC involves the minimization of multiple objectives, resulting in the potential concerns of *feature suboptimality* and *class dependence*. Consequently, no effective learning scheme has been established to improve NPSVCs' performance through representation learning, especially deep learning. To break this bottleneck, we develop NPSVC++ based on multiobjective optimization, enabling the seamless learning of NPSVC and its features. By pursuing Pareto optimality, NPSVC++ theoretically ensures feature optimality across different classes, thus effectively overcoming these two issues mentioned above. A general learning procedure via duality optimization is proposed, based on which we provide two applicable instances, K-NPSVC++ and D-NPSVC++. Moreover, the convergence analysis of the proposed algorithm is provided. The experiments show their superiority over the existing methods and verify the efficacy of NPSVC++.

Index Terms—Deep learning, kernel method, multiobjective optimization, nonparallel support vector classifiers (NPSVCs).

I. INTRODUCTION

SUPPORT vector machine (SVM) [1] has been investigated for decades as a powerful classification algorithm. The elegant margin theory behind SVM assures its powerful generalization ability [2]. Due to its remarkable performance, SVM is used for various applications, such as text classification [3] and image classification [4]. Numerous variants emerged in response to the advent of SVM, becoming a new development trend. For example, least-squares SVM [5] is a representative variant of SVM still studied today [6]. Recently, Chan et al. [7] improved SVM with the additive kernel and

Received 8 October 2024; revised 7 April 2025, 23 July 2025, 25 September 2025, and 5 November 2025; accepted 27 January 2026. Date of publication 27 February 2026; date of current version 2 September 2026. This work was supported in part by the Natural Science Foundation of China under Grant 62476175 and Grant 62272319; in part by the Natural Science Foundation of Guangdong Province under Grant 2024A1515011637 and Grant 2023B1212060076; and in part by Shenzhen Science and Technology Program under Grant JCYJ20220818095803007, Grant JCYJ20240813142206009, and Grant JCYJ20250604145234045. (Corresponding author: Zhihui Lai.)

Junhong Zhang, Zhihui Lai, and Guangfei Liang are with the Computer Vision Institute, College of Computer Science and Software Engineering, Shenzhen University, Shenzhen 518060, China, and also with Guangdong Provincial Key Laboratory of Intelligent Information Processing, Shenzhen 518060, China (e-mail: apple_zjh@163.com; lai_zhi_hui@163.com; gf_liang_study@163.com).

Jie Zhou is with the National Engineering Laboratory for Big Data System Computing Technology, Shenzhen University, Shenzhen, Guangdong 518060, China (e-mail: jie_jpu@163.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TNNLS.2026.3661343>, provided by the authors.

Digital Object Identifier 10.1109/TNNLS.2026.3661343

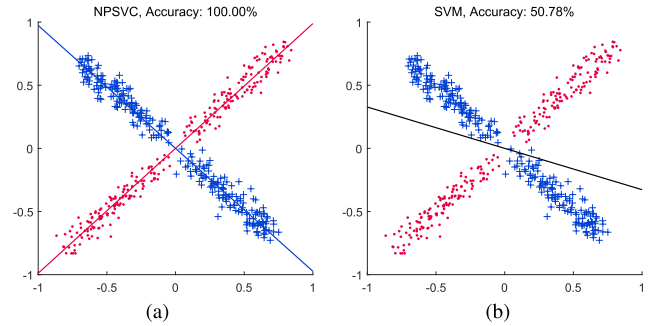


Fig. 1. Comparison of NPSVC and SVM on cross plane data. (a) Linear NPSVC can classify the data perfectly. (b) Linear SVM fails to handle it.

significantly reduced the computation burden. On the other hand, different novel loss functions are integrated into SVM to enhance robustness [8], [9]. The nonparallel classifiers, as a collection of improved SVMs, have been gaining the interest of researchers. The basic goal of these methods is to seek a series of hyperplanes in the data space, such that each hyperplane is close to the corresponding class and far from the others. Each class is associated with the hyperplane closest to it. In this way, the new data are categorized into the class of the respective nearest hyperplane. One of the early works, Mangasarian and Wild [10] realized such a concept by tackling an eigensystem, which is yet computationally expensive and is rarely applied. More importantly, Jayadeva et al. [11] proposed twin SVM (TWSVM) that is more efficient and practical, and it establishes the learning scheme that minimizes similarity and dissimilarity loss functions to learn the classification hyperplanes. The subsequent nonparallel classifiers, such as [12], [13], [14], [15], and [16], mostly inherit this scheme. The distinctions among these methods pertain to their loss functions and optimization algorithms. We categorize them into a family of nonparallel classifiers, namely, nonparallel support vector classifiers (NPSVCs), in this article. NPSVC and SVM are significantly different in terms of learning scheme and classification strategy. The former matches the data to the nearest hyperplane, while the latter separates the data from each class with the largest margin. An intuitive example to illustrate the difference between SVM and NPSVC is the “cross plane” problem [11]. As shown in Fig. 1, linear SVM fails to classify the data due to the nonlinearity of the data, while linear NPSVC achieves perfect classification. This result demonstrates the superiority of NPSVC in handling complex data. Indeed, NPSVC has become a more flexible

and powerful alternative to SVM in many scenarios [13], [17], [18].

Representation learning is also one of the most important techniques in machine learning. Typically, data are depicted as vectors in a Euclidean space, referred to as representations, or features. The quality of the features has a substantial impact on the downstream tasks. A basic consensus is that the discriminative features will improve the performance of classifiers [19]. However, it is challenging to discover the underlying features of the intricate data. A widely approved solution is deep learning, which leverages the powerful expressiveness of deep neural networks (DNNs) to achieve end-to-end feature learning and classification. DNNs with diverse architectures have demonstrated outstanding results in classification tasks, such as convolution neural network (CNN) [20], [21], [22] and Transformer [23], [24]. Moreover, the classifier (or classification layer) plays an imperative role in determining the learning criteria of DNNs. Softmax regression is a typical choice for classification in deep learning, which is associated with cross-entropy loss [25]. SVM is usually used as the downstream classifier in deep learning, and the existing DNNs with SVM have shown promising classification performance [26], [27].

The success of deep SVM motivates us to explore the potential of NPSVCs in the context of deep learning. Although NPSVC has been extensively studied for a decade, there are still few works that investigate the deep extension of NPSVC. Unlike SVM and softmax regression, NPSVC involves multiple minimization problems, from which we identify two key issues in developing the deep extensions for NPSVCs.

- 1) *Feature Suboptimality*: The input features are probably not optimal for NPSVC's classification.
- 2) *Class Dependence*: The relationship between classes is neglected during optimization.

Despite some attempts for deep NPSVCs, the two issues are still not addressed. For example, the deep NPSVC model proposed by Xie et al. [28] is not end-to-end, and thus, the induced features are still suboptimal for classification (*feature suboptimality*); Li and Yang [29] and Kumar et al. [30] proposed DNNs based on TWSVM, but they neglect the interaction between classes and fail to achieve the tradeoff among multiple objectives (*ignoring class dependence*). To implement the end-to-end learning of NPSVC and guarantee its optimal representations, it is necessary to explore a new learning strategy to address the above two issues. We list the contributions of this article as follows.

- 1) We develop a novel seamless learning framework, called NPSVC++, which enables the incorporation of NPSVCs with representation learning through multi-objective optimization. NPSVC++, for the first time, reforges the learning paradigm of NPSVCs and breaks their limitations regarding feature suboptimality and class dependence, making these classifiers more flexible and applicable.
- 2) We adopt an iterative duality optimization method to achieve Pareto optimality. As such, in contrast to the previous methods, NPSVC++ theoretically ensures the optimality of the learned features across different objec-

tives. The convergence of the proposed method is also proven under mild conditions.

- 3) Two realizations of NPSVC++ are proposed: K-NPSVC++ with kernel setting and D-NPSVC++ for deep learning. The former is suitable for small datasets ($\leq 2 \times 10^4$ samples), while the latter is more efficient for larger scale datasets. Numerical experiments verify their effectiveness.

The rest of this article is organized as follows. Section II provides the frequently used notations and the background of NPSVC. The framework of NPSVC++ is introduced in Section III, based on which we present the two instances, K-NPSVC++ and D-NPSVC++, in Section IV. The experimental results are shown in Section VI, and the conclusion is drawn in Section VII.

II. PRELIMINARIES

In this section, we provide the frequently used notations and briefly review the related works.

A. Notations

We use bolded letters like $\mathbf{a}$ (or $\mathbf{A}$) to represent vectors (or matrices). For a matrix $\mathbf{A}$, its transpose is denoted as $\mathbf{A}^\top$. Specially, $\mathbf{A}^{-\top}$ is the transpose of $\mathbf{A}^{-1}$ if $\mathbf{A}$ is invertible. $\text{Tr}(\mathbf{A})$ is the trace of matrix $\mathbf{A}$, i.e., the sum of diagonal elements of $\mathbf{A}$. The $\mathbf{1}$ and $\mathbf{0}$ are the all-ones and all-zeros vectors in the proper dimension, respectively. Denote shorthands $[n] = \{1, 2, \dots, n\}$ for any $n \in \mathbb{Z}^+$ and $[\cdot]_+ \triangleq \max(0, \cdot)$. For a classification task, let $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n] \in \mathbb{R}^{m \times n}$ be the training samples, where $\mathbf{x}_i \in \mathbb{R}^m$ is the i th sample, and $y_i \in \mathcal{Y}$ denote its label. We denote the hypothesis function using $f(\cdot) : \mathbb{R}^m \mapsto \mathbb{R}$. Let $\mathbb{R}_+^m = \{\mathbf{x} \in \mathbb{R}^m : x_i \geq 0, \forall i \in [m]\}$. For two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^m$, we say $\mathbf{a} \geq \mathbf{b}$ if $\mathbf{a} - \mathbf{b} \in \mathbb{R}_+^m$, and oppositely, $\mathbf{a} \leq \mathbf{b}$ if $\mathbf{b} - \mathbf{a} \in \mathbb{R}_+^m$. Define the Euclidean projection of $\mathbf{x}$ to a set $\mathcal{S}$ as $\pi_{\mathcal{S}}(\mathbf{x}) = \min_{\mathbf{y} \in \mathcal{S}} \|\mathbf{x} - \mathbf{y}\|^2$.

B. Support Vector Machine

Consider a binary-class dataset and the linear hypothesis $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$, and thus, $f(\mathbf{x}) = 0$ denotes a hyperplane in $\mathbb{R}^m$ with the normal vector $\mathbf{w}$ and bias b . SVM explores such a hyperplane that maximizes the margin between the two classes, and it is formulated with the following problem:

$$\min_{\mathbf{w}} c \sum_{i=1}^n [1 - y_i f(\mathbf{x}_i)]_+ + \frac{1}{2} \|\mathbf{w}\|^2 \quad (1)$$

where $c > 0$ is the penalty parameter. Generally, (1) is optimized via its dual problem, i.e., the following quadratic programming problem (QPP):

$$\min_{\boldsymbol{\alpha}} \frac{1}{2} \boldsymbol{\alpha}^\top \mathbf{Q} \boldsymbol{\alpha} - \mathbf{1}^\top \boldsymbol{\alpha}, \quad \text{s.t. } 0 \leq \alpha_i \leq c, \mathbf{y}^\top \boldsymbol{\alpha} = 0 \quad (2)$$

where $\boldsymbol{\alpha} \in \mathbb{R}^n$ is the nonnegative Lagrange multipliers, and the matrix $\mathbf{Q}$ is defined as $Q_{ij} = y_i y_j \mathbf{x}_i^\top \mathbf{x}_j$. Sequential minimization optimization (SMO) is usually used to solve this problem [31]. With the optimal $\boldsymbol{\alpha}$, $\mathbf{w}$ can be computed via $\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$ and $b = (1/|S|) \sum_{i \in S} (y_i - \mathbf{w}^\top \mathbf{x}_i)$, where $S = \{i : 0 < \alpha_i < c\}$

TABLE I
REPRESENTATIVE METHODS OF NPSVC. s IS THE MARGIN SIZE,
USUALLY $s = 1$, AND ϵ, τ, γ , AND t ARE THE HYPERPARAMETERS

Methods	$\ell_s(a)$	$\ell_d(a)$
TWSVM [11]	$a^2/2$	$[s - a]_+$
LS-TWSVM [12]	$a^2/2$	$(s - a)^2/2$
NPSVM [33]	$[a - \epsilon]_+$	$[s - a]_+$
pin-TWSVM [14]	$a^2/2$	$\begin{cases} s - a & a \leq s \\ -\tau(s - a) & a > s \end{cases}$
CTWSVM [34]	$\min(a , \epsilon)$	$\min([s - a]_+, \epsilon)$
Corr-TWSVM [35]	$1 - e^{-\gamma a }$	$[s - a]_+$
RMTBSVM [36]	$\min(a , \epsilon)$	$[s - a]_+$

is the index set of the data points lying in the hyperplane $f(\mathbf{x}) = \pm 1$ (known as support vectors). In the testing phase, the label of the new data $\mathbf{x}'$ is predicted by

$$h(\mathbf{x}') = \text{sign}(\mathbf{w}^\top \mathbf{x}' + b) = \text{sign}\left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i^\top \mathbf{x}' + b\right). \quad (3)$$

C. Nonparallel Support Vector Classifier

NPSVCs indicate a series of improved SVM models. Nevertheless, their learning scheme differs from SVM and many mainstream classifiers like softmax regression. NPSVCs classify the data with multiple nonparallel hyperplanes. For binary-class data, $\mathcal{Y} = \{-1, 1\}$, an NPSVC seeks a pair of hyperplanes, namely, the positive hyperplane $f_+(\mathbf{x}) = \mathbf{w}_+^\top \mathbf{x} + b_+ = 0$ and negative hyperplane $f_-(\mathbf{x}) = \mathbf{w}_-^\top \mathbf{x} + b_- = 0$. The data will be classified into the class corresponding to the nearest hyperplane. Thus, binary-class NPSVC uses the following prediction rule:

$$h(\mathbf{x}') = \text{sign}\left(\frac{|f_-(\mathbf{x}')|}{\|\mathbf{w}_-\|} - \frac{|f_+(\mathbf{x}')|}{\|\mathbf{w}_+\|}\right).$$

This means the positive hyperplane should be close to the positive samples ($y_i = 1$) and far away from negative samples ($y_i = -1$), and the negative hyperplane behaves oppositely. As an intuitive explanation, each hyperplane draws a portrait of each class, and each class is attached to the closest hyperplane. To this end, NPSVC consists of two problems that optimize the positive and negative hyperplanes

$$\begin{aligned} \min_{f_+} R(f_+) + \sum_{y_i > 0} \ell_s(f_+(\mathbf{x}_i)) + c \sum_{y_i < 0} \ell_d(f_+(\mathbf{x}_i)) \\ \min_{f_-} R(f_-) + \sum_{y_i < 0} \ell_s(f_-(\mathbf{x}_i)) + c \sum_{y_i > 0} \ell_d(f_-(\mathbf{x}_i)) \end{aligned} \quad (4)$$

where $c > 0$ is the hyperparameter, $R(\cdot)$ is a regularizer, and ℓ_s and ℓ_d denote the similarity and dissimilarity losses, respectively. TWSVM [11] is one of the most popular models of NPSVC, which utilizes $\ell_s(a) = a^2/2$ and $\ell_d(a) = [1 - a]_+$. Table I lists some typical NPSVC methods and their loss functions.

Now, we consider the multiclass setting with $\mathcal{Y} = [K]$, where $K \geq 2$ is the number of classes. This article focuses on the widely used one-versus-rest (OVR) multiclass NPSVC [32]. It seeks K hyperplanes such that the samples from l th class are close to l th hyperplane $f_l(\mathbf{x}) = 0$, where $f_l(\mathbf{x}) =$

$\mathbf{w}_l^\top \mathbf{x} + b_l$, and other samples are far from it. Analogous to the binary-class case, the prediction rule is to match the nearest hyperplane

$$h(\mathbf{x}') = \arg \min_{l \in [K]} \frac{|f_l(\mathbf{x}')|}{\|\mathbf{w}_l\|}. \quad (5)$$

Accordingly, it formulates the training criterion as

$$\min_{f_l} R(f_l) + \sum_{y_i = l} \ell_s(f_l(\mathbf{x}_i)) + c \sum_{y_i \neq l} \ell_d(f_l(\mathbf{x}_i)), \quad l \in [K]. \quad (6)$$

It should be noted that (6) consists of K problems. Unless explicitly stated otherwise, the remaining analysis in this work uses the OVR strategy for the multiclass setting.

SVM and NPSVC have some similar properties to an extent (e.g., loss functions), but their classification mechanisms are substantially unlike. There are two major aspects. First, the roles of hyperplanes are distinct in SVM and NPSVC. This is also reflected in the prediction rule (3) and (5). The hyperplanes in SVM are “class boundaries” used to separate multiple classes, while in NPSVC, the hyperplanes act as “class profiles” so that the data can be categorized into the nearest hyperplane. Second, they apply different learning criteria. Intuitively, SVM finds the hyperplanes maximizing the interclass margin. NPSVC takes into account both intraclass distance and interclass distance. By Fisher’s discriminant criterion, NPSVC is deemed more powerful than SVM, which is also supported by the existing studies [36], [37], [38]. The most intuitive example is their performance on XOR data; see [17] for details. It is shown that linear NPSVC can perfectly deal with the XOR data, but linear SVM fails to do so (see also Fig. 1).

D. Representation Learning

Representation learning aims to obtain the proper features of raw data for some specific purposes, e.g., dimension reduction and adaptation to the downstream tasks. Subspace learning includes a series of the most basic approaches that project the data into a specific subspace. Principal component analysis (PCA) [39], fisher discriminative analysis (FDA) [39], and locality preserving projection (LPP) [40] are typical ones. Despite their efficacy, they are usually task-independent and thus cannot guarantee that the learned features are suitable for the downstream tasks, e.g., classification. This is also related to the “feature suboptimality” issue discussed later. To this end, researchers have explored novel subspace learning methods to find suitable features for SVM classification [19], [41], [42].

Deep learning provides an end-to-end framework to learn features and classifiers simultaneously. A well-known example is CNN for image classification [20], [22], consisting of multiple convolution layers for feature extraction and a softmax layer for classification. On the one hand, the quality of features is enhanced along with various advanced modules and architectures, e.g., residual connection [21] and attention [23]. On the other hand, the learning criterion is crucial for DNN as it drives the training of the whole network. For example, cross-entropy is a common choice for classification [25], which is improved by Wang et al. [43] and Deng et al. [44] using the interclass margin. As a famous classifier, SVM also motivates

a series of novel criteria of DNNs [27], [45]. Li and Yang [29] leveraged the criterion of TWSVM and proposed its DNN extension, making DNNs benefit from the merit of NPSVC. However, it disregards the “class dependence” issue in NPSVC, which will be discussed in depth in Section III-A.

III. METHODOLOGY OF NPSVC++

As suggested before, NPSVC shows superiority for classifying intricate data compared to SVM. Therefore, it is valuable to explore the potential of NPSVC. However, few studies have been conducted to extend NPSVC with representation learning, which also limits the application of NPSVC in deep learning. To this end, this article aims to bridge this gap by proposing NPSVC++. In this section, we elucidate on the motivation and the methodology of NPSVC++.

A. Motivation: Issues in NPSVCs

The current works on nonparallel classifiers mostly follow the framework (6) and exploit various loss functions to improve the models’ capacity and robustness. However, there are two unsolved issues that hinder the further development of NPSVCs.

- 1) *Feature Suboptimality*: The basic assumption in NPSVC is that the data of each class can be illustrated by the associated hyperplanes. Thus, the quality of the data representations, i.e., features, is typically influential. It poses the problem of how to find high-quality features to enhance NPSVCs. To this end, feature selection approaches were proposed [46], [47]. Alternatively, Xie et al. [48] weight the features based on information gain. However, they primarily focus on filtering the unimportant features of the raw data rather than improving data separability. Recently, the feature enhancement method based on granular ball was proposed to improve TWSVM (see [49]). On the other hand, Xie et al. [28] extract the feature with a pretrained autoencoder. Although informative features are obtained, these features lack supervision information and thus may be unsuitable for NPSVC. In fact, these methods fail to establish a criterion to optimize the feature along with the classifier, resulting in feature suboptimality.
- 2) *Class Dependence*: The second issue in NPSVC is how to make the learning be “class-collaborative.” We have mentioned that the K problems in NPSVC (6) are optimized independently. In other words, the learning of each hyperplane is agnostic to others. Hence, the correlation between different classes is ignored, which likely results in inadequate performance. For example, consider an image classification task with three classes: “rabbit,” “tiger,” and “leopard.” Intuitively, tigers and leopards likely have some similar characteristics as they both belong to the Felidae family, while NPSVCs may fail to recognize this distinction across different classes. We argue that these two issues are systematic in NPSVC. In order to address “feature suboptimality,” a natural idea is to train a feature extractor along with the classifier, which captures the inherent dependence

between different classes. This motivates [29] to build a DNN of TWSVM with all classes sharing a single encoder (i.e., feature extractor). Similarly, the recent work [30] builds an attention-based DNN for least-square TWSVM. However, they still have a common problem. In fact, the parameters of this encoder are shared by all objectives, yielding an intricate dependence across all classes. However, Li and Yang [29] and Jimenez-Castaño et al. [50] disregard how to deal with such dependence, and thus fail to reach the tradeoff among different objectives. Therefore, the optimality of classifiers and features is uncertain. This falls back into the second issue, “class dependence,” which is an unavoidable problem in multiclass NPSVC since it involves multiple optimization problems. It implies that the approach of simply combining DNNs cannot be duplicated in NPSVC. Unlike the seamless multiclass SVM model like [51] that optimizes a single objective, there is still a lack of a principled approach to managing class dependence in NPSVC. Hence, optimizing the dependent objectives of NPSVC remains an open problem. In a nutshell, a question emerges: “How to obtain the optimal features and classifiers for all classes in NPSVC?” The key to this question is the optimality of multiple dependent objectives. Accordingly, we propose a framework that composes representation learning and NPSVC based on multiobjective optimization, namely, NPSVC++.

B. NPSVC++: Modeling With Pareto Optimality

Recall the optimization model of NPSVC (6). Denote J_l^{NP} as the l th objective function of (6). Then, NPSVC can be reformulated as a multiobjective optimization problem

$$\min_{\{\mathbf{w}_l, b_l\}_{l=1}^K} \mathbf{J} \triangleq [J_1^{\text{NP}}(\mathbf{w}_1, b_1), \dots, J_K^{\text{NP}}(\mathbf{w}_K, b_K)] \quad (7)$$

which means that all objective functions J_l^{NP} are minimized simultaneously. Indeed, in NPSVC, each objective J_l^{NP} has no shared variables, meaning that the objectives are mutually independent. Thus, problem (7) is equivalent to (6), which means each objective can be minimized independently. Alternatively, we consider a more general multiobjective optimization framework with shared variables

$$\min_{\boldsymbol{\theta}} \mathbf{J}(\boldsymbol{\theta}) \triangleq [J_1(\boldsymbol{\theta}_1, \boldsymbol{\theta}_{\text{sh}}), \dots, J_K(\boldsymbol{\theta}_K, \boldsymbol{\theta}_{\text{sh}})] \quad (8)$$

where $\boldsymbol{\theta}_l$ denotes the variables specific to the l th class and $\boldsymbol{\theta}_{\text{sh}}$ represents the shared variables for all classes. Let $\boldsymbol{\theta} = \{(\boldsymbol{\theta}_l)_{l=1}^K, \boldsymbol{\theta}_{\text{sh}}\}$ denote all optimization variables.

We now formulate NPSVC++ following the framework (8). First, similar to NPSVC, the objective function of each class J_l is formulated as

$$J_l \triangleq \sum_{y_i=l} \ell_s(f_l(\mathbf{x}_i)) + c \sum_{y_i \neq l} \ell_d(f_l(\mathbf{x}_i)) + R(f_l) \quad (9)$$

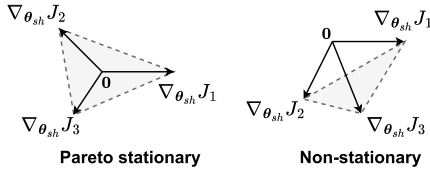


Fig. 2. Illustration of Pareto stationarity in $\mathbb{R}^2$ and three objectives. The gray area is the convex hull of $\{\nabla_{\theta_{sh}} J_l\}_{l=1}^K$, and $\mathbf{0}$ is the zero vector.

where, distinctive from NPSVC, the hypothesis function of NPSVC++ is defined as¹

$$f_l(\mathbf{x}) \triangleq \langle \mathbf{w}_l, \boldsymbol{\phi}(\mathbf{x}) \rangle + \mathbf{v}_l^\top \mathbf{z}(\mathbf{x}). \quad (10)$$

Here, $\mathbf{w}_l$ and $\mathbf{v}_l$ are the class-specific weights, $\boldsymbol{\phi}(\cdot)$ is a prior transformation, and $\mathbf{z}(\cdot)$ is the encoder shared by all classes. Hypothesis (10) exhibits a skip connection structure, which aims to preserve the raw information $\boldsymbol{\phi}(\mathbf{x})$ and avoid degradation [52]. Recalling (8), in NPSVC++, the class-specific variable is $\boldsymbol{\theta}_l = (\mathbf{w}_l, \mathbf{v}_l)$ and the shared variable $\boldsymbol{\theta}_{sh}$ is the parameters of encoder $\mathbf{z}(\cdot)$. The major challenge in NPSVC++ stems from the shared variable $\boldsymbol{\theta}_{sh}$. Since the shared variable affects all objective functions, its optimization would entail intricate interactions or potential conflicts among various objectives [53]. The fundamental goal is to achieve a compromise among different objectives. Therefore, NPSVC++ endeavors to pursue the *Pareto optimal solution*. We first introduce the definition.

Definition 1 (Pareto Optimality [54]): A solution $\boldsymbol{\theta}^*$ to (8) is Pareto optimal, if there exists no $\boldsymbol{\theta}$ such that $J_l(\boldsymbol{\theta}) \leq J_l(\boldsymbol{\theta}^*)$ for all $l \in [K]$ and $\mathbf{J}(\boldsymbol{\theta}) \neq \mathbf{J}(\boldsymbol{\theta}^*)$.

Pareto optimality means there exists no better solution that can further improve all objectives. Pareto optimality is the key to resolving the issues of feature suboptimality and class dependence in NPSVC++ since it naturally defines the optimality among all objectives. Generally, Pareto optimality is achieved by the Pareto stationarity stated in the following.

Definition 2 (Pareto Stationarity [55]): A solution to (8) is Pareto stationary if existing nonzero $\boldsymbol{\tau} \in \mathbb{R}_+^K$ satisfies

$$\sum_{l=1}^K \tau_l \nabla_{\theta_{sh}} J_l(\boldsymbol{\theta}_l, \boldsymbol{\theta}_{sh}) = \mathbf{0}, \quad \sum_{l=1}^K \tau_l = 1, \quad \tau_l \geq 0. \quad (11)$$

Pareto stationarity is a necessary condition of Pareto optimality [55]. Intuitively, (11) indicates that $\mathbf{0}$ is in the convex hull of $\{\nabla_{\theta_{sh}} J_l\}_{l=1}^K$. As shown in Fig. 2, (11) means that the gradients of the objectives are balanced with each other, and no movement can reduce all objectives, implying Pareto optimality of $\boldsymbol{\theta}_{sh}$ and the feature encoder. To see this, take $\mathbf{g}$ from the convex hull of $\{\nabla_{\theta_{sh}} J_l\}_{l=1}^K$, and we must have $\langle \mathbf{g}, \nabla_{\theta_{sh}} J_{l'} \rangle \leq 0$ for at least one $l' \in [K]$; otherwise, we obtain $\langle \mathbf{g}, \sum_{l=1}^K \tau_l \nabla_{\theta_{sh}} J_l \rangle > 0$, contradicting (11). This implies $\mathbf{g}$ in the convex hull conflicts with the descent direction of at least one objective. In the nonstationary case of Fig. 2, we can find a direction that reduces all objectives (e.g., $\mathbf{g} = -\nabla_{\theta_{sh}} J_3$).

It is essential to seek an appropriate Pareto stationary point and the corresponding $\boldsymbol{\tau}$. Inspired by [56] and [57], we

¹We omit the bias term b_l for convenient discussion. One can append an additional 1 after features to implicitly include the bias.

consider the weighted Chebyshev decomposition problem of (7)

$$\min_{\rho, \boldsymbol{\theta}} \rho, \quad \text{s.t. } J_l(\boldsymbol{\theta}_l, \boldsymbol{\theta}_{sh}) \leq \rho \quad \forall l \in [K]. \quad (12)$$

Intuitively, problem (12) minimizes all objective functions via suppressing their upper bound ρ . In the following theorem, we show that the dual problem of (12) presents a doable criterion to achieve the Pareto stationary solution.

Theorem 1: The dual problem of (12) is

$$\begin{aligned} \max_{\boldsymbol{\tau}} \min_{\boldsymbol{\theta}} \sum_{l=1}^K \tau_l J_l(\boldsymbol{\theta}_l, \boldsymbol{\theta}_{sh}) \\ \text{s.t. } \sum_{l=1}^K \tau_l \nabla_{\theta_{sh}} J_l = \mathbf{0}, \quad \mathbf{1}^\top \boldsymbol{\tau} = 1, \quad \tau_l \geq 0 \end{aligned} \quad (13)$$

where $\boldsymbol{\tau} \in \mathbb{R}_+^K$ denotes the Lagrange multipliers.

The proof is shown in the Appendix in the supplementary material. The key to training NPSVC++ is to solve the max-min dual problem (13), leading to the following iterative optimization procedure:

$$\boldsymbol{\theta}_{l+1} = \arg \min_{\boldsymbol{\theta}} \boldsymbol{\tau}_l^\top \mathbf{J}(\boldsymbol{\theta}) \quad (14)$$

$$\boldsymbol{\tau}_{l+1} = \arg \max_{\boldsymbol{\tau} \in \Delta} \boldsymbol{\tau}^\top \mathbf{J}(\boldsymbol{\theta}_{l+1}), \quad \text{s.t. } \nabla_{sh} \boldsymbol{\tau} = \mathbf{0} \quad (15)$$

where matrix $\nabla_{sh} = [\nabla_{\theta_{sh}} J_1, \dots, \nabla_{\theta_{sh}} J_K]$, and $\Delta = \{\boldsymbol{\tau} \in \mathbb{R}_+^K : \mathbf{1}^\top \boldsymbol{\tau} = 1\}$ denotes the probability simplex. The basic structure of NPSVC++ is illustrated in Fig. 3.

IV. INSTANTIATION

In this section, we will elaborate on the learning of NPSVC++ with two typical settings, i.e., kernel machine (K-NPSVC++) and DNNs (D-NPSVC++).

A. NPSVC++ as Kernel Machine: K-NPSVC++

Let prior $\boldsymbol{\phi}(\cdot)$ be the induced mapping of kernel $\mathcal{K}(\cdot, \cdot)$, which is assumed positive-definite, and $\mathcal{H}$ is the corresponding reproducing kernel Hilbert space (RKHS). Denote $\langle \cdot, \cdot \rangle$ the inner product operator in $\mathcal{H}$. We define Stiefel manifold $\text{St}(\mathcal{H}, d) = \{\mathbf{P} \in \mathcal{H}^d : \mathbf{P}^\top \mathbf{P} = \mathbf{I}\}$. In some unambiguous contexts, we directly write the notation St for simplification.

In the following discussion, we denote $\boldsymbol{\phi}(\mathbf{X}) = [\boldsymbol{\phi}(\mathbf{x}_i)]_{i=1}^n$ the feature matrix, and $\mathbf{K} \in \mathbb{R}^{n \times n}$ the kernel matrix with each entry $K_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$. We parameterize $\mathbf{z}(\mathbf{x}) = \mathbf{P}^\top \boldsymbol{\phi}(\mathbf{x})$ with a projection $\mathbf{P} \in \text{St}(\mathcal{H}, d)$. In this way, $\mathbf{P}$ defines an orthonormal system to span a subspace in the RKHS of $\mathcal{K}$. The orthonormal constraint of $\mathbf{P}$ ensures that the subspace is rank- d and isometric, preventing it from degenerating into a trivial subspace. This constraint is also widely used in the existing subspace learning methods such as [41] and [42]. Then, we have

$$\begin{aligned} f_l(\mathbf{x}) &= \langle \mathbf{w}_l, \boldsymbol{\phi}(\mathbf{x}) \rangle + \mathbf{v}_l^\top \mathbf{z}(\mathbf{x}) \\ &= \langle \mathbf{w}_l + \mathbf{P} \mathbf{v}_l, \boldsymbol{\phi}(\mathbf{x}) \rangle = \langle \mathbf{u}_l, \boldsymbol{\phi}(\mathbf{x}) \rangle \end{aligned} \quad (16)$$

where we let $\mathbf{u}_l = \mathbf{w}_l + \mathbf{P} \mathbf{v}_l$. This substitution will simplify the subsequent optimization, and thus, the optimization variables become $\boldsymbol{\theta} = \{\mathbf{U}, \mathbf{V}, \mathbf{P}\}$, where $\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_K] \in \mathcal{H}^K$ and

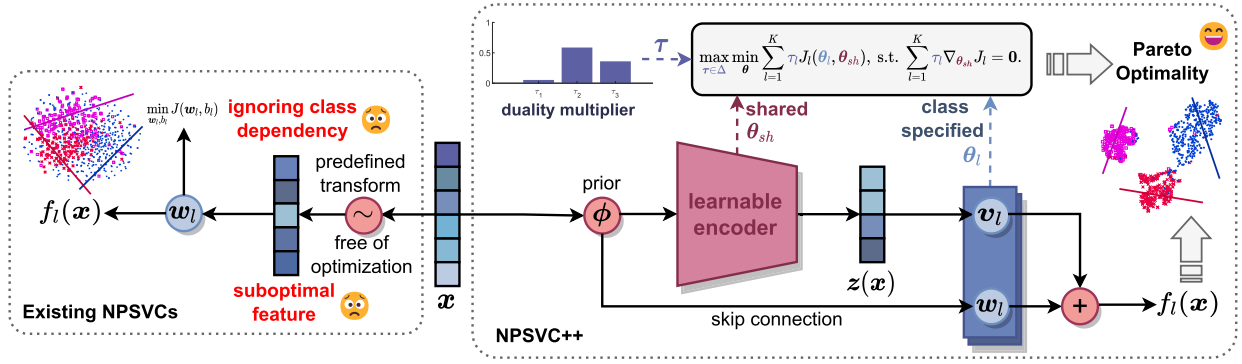


Fig. 3. NPSVC usually uses learning-free feature transformation and optimizes the hyperplane of each class independently, resulting in feature suboptimality and disregarding class dependence (left). NPSVC++ stems from multiobjective optimization, whose goal is to achieve Pareto optimality (right). Thus, feature optimality across different classes is ensured, tackling the issues of NPSVCs.

$V = [v_1, \dots, v_K] \in \mathbb{R}^{d \times K}$. In this instance, we inherit the loss of TWSVM, i.e., $\ell_s(a) = a^2/2$ and $\ell_d(a) = [1 - a]_+$, and use the following regularizer:

$$R(w_l, v_l, z) = \frac{r_1}{2} \|w_l\|^2 + \frac{r_2}{2} \|v_l\|^2 + \frac{\mu}{4} \sum_{i,j} \|z(x_i) - z(x_j)\|^2 G_{ij}$$

where r_1, r_2 , and μ are the positive hyperparameters. The third term, known as the manifold regularization [58], can illustrate the intraclass discrepancy of the data in the subspace as proposed in [40]. We regularize P with this term to obtain a subspace with high similarity within each class, benefiting the subsequent classifier. The graph adjacency matrix $G \in \mathbb{R}^{n \times n}$ describes the pairwise relation of the data. Denote D a diagonal matrix with the i th diagonal element $D_{ii} = \sum_{j=1}^n G_{ij}$ and $L = D - A$ the Laplacian matrix. Then, the l th objective function is reformulated as

$$J_l = \frac{1}{2} \sum_{y_i \neq l} \langle u_l, \phi(x_i) \rangle^2 + c \sum_{y_i \neq l} [1 - \langle u_l, \phi(x_i) \rangle]_+ + \frac{r_1}{2} \|u_l - P v_l\|^2 + \frac{r_2}{2} \|v_l\|^2 + \frac{\mu}{2} \text{Tr}(Z L Z^T)$$

where $Z = P^T \phi(X)$ and the regularization of w is written as $\|w_l\|^2 = \|u_l - P v_l\|^2$. By representer theorem [39], there exists $A = [\alpha_{ij}] \in \mathbb{R}^{n \times d}$, where $A^T K A = I$ and $B = [\beta_{il}] \in \mathbb{R}^{n \times K}$, such that

$$u_l^* = \sum_{i=1}^n \beta_{il} \phi(x_i), \quad p_j^* = \sum_{i=1}^n \alpha_{ij} \phi(x_i). \quad (17)$$

Then, the objective function is reformulated as

$$J_l = \frac{1}{2} \|K_l^T \beta_l\|^2 + c \sum_{y_i \neq l} [1 - \beta_l^T \kappa_i]_+ + \frac{r_2}{2} \|v_l\|^2 + \frac{r_1}{2} \|\phi(X) \beta_l - \phi(X) A v_l\|^2 + \frac{\mu}{2} \text{Tr}(A^T K L K A)$$

where κ_i is the i th column of K and $K_l = [\kappa_i]_{y_i \neq l}$. We employ a further substitution to simplify the computation. Instead of directly optimizing A and B , we optimize $\hat{P} = \Psi^T A$ and $\hat{U} = \Psi^T B$, where Ψ satisfies the decomposition $K = \Psi \Psi^T$. Matrix Ψ^T can be seen as the empirical version of $\phi(X)$, and thus, $\hat{P} \in \mathbb{R}^{n \times d}$ and $\hat{U} \in \mathbb{R}^{n \times K}$ serve as the empirical version

of $P \in \mathcal{H}^d$ and $U \in \mathcal{H}^K$, respectively. We generally set Ψ as the Cholesky factor of K , which ensures the invertibility of Ψ . In this way, the substitution is bijective and $A = \Psi^{-T} \hat{P}$ and $B = \Psi^{-T} \hat{U}$. This trick avoids handling the tricky constraint $A^T K A = I$. Instead, the constraint becomes $\hat{P}^T \hat{P} = I$, or equivalently, $\hat{P} \in \text{St}(\mathbb{R}^n, d)$, allowing us to employ existing efficient Stiefel optimization methods.

Remark 1: A common issue of kernel methods is their poor scalability, which is generally associated with $O(n^3)$ time complexity. To alleviate this issue, one can use the Nyström approximation method to reduce the computational cost when n is large. Specifically, we can set $\Psi = K_{:,S} K_{S,S}^{-1/2}$ such that $K \approx \Psi \Psi^T$, where S is the index set of sampled columns, and $K_{:,S}$ is the submatrix of K with columns indexed by S . As suggested by Rudi et al. [59], a proper choice of $m = O(\sqrt{n})$ can achieve a good approximation while ensuring efficiency, resulting in $O(n^{3/2})$ time complexity and $O(n)$ space complexity.

Now, with a series of transformations, the eventual objective function becomes

$$J_l = \frac{1}{2} \|\Psi_l \hat{u}_l\|^2 + c \sum_{y_i \neq l} [1 - \hat{u}_l^T \psi_i]_+ + \frac{r_2}{2} \|v_l\|^2 + \frac{r_1}{2} \|\hat{u}_l - \hat{P} v_l\|^2 + \frac{\mu}{2} \text{Tr}(\hat{P}^T \Psi^T L \Psi \hat{P})$$

where $\psi_i \in \mathbb{R}^n$ is the i th row of Ψ . We apply the alternative minimization strategy to implement the minimization step (14), which is elaborated as follows.

U Step: In this step, we fix $\hat{V}$ and $\hat{P}$ to optimize $\hat{u}_l$ for each $l \in [K]$. We first compute three auxiliary matrices

$$\begin{aligned} Q_{\Psi, \Psi}^{(l)} &= \frac{1}{r_1} [I - \Psi_l^T (r_1 I + \Psi_l \Psi_l^T)^{-1} \Psi_l] \\ Q_{K, \Psi}^{(l)} &= \Psi_{-l}^T Q_{\Psi, \Psi}^{(l)}, \quad Q_{K, K}^{(l)} = \Psi_{-l}^T Q_{\Psi, \Psi}^{(l)} \Psi_{-l} \end{aligned} \quad (18)$$

where $\Psi_l = [\psi_i]_{y_i \neq l}$ denotes those rows in Ψ of l th class, and Ψ_{-l} corresponds to the other rows. Then, we have the following closed-form solution:

$$\hat{u}_l = r_1 Q_{\Psi, \Psi}^{(l)} \hat{P} v_l + Q_{K, \Psi}^{(l)} \lambda_l \quad (19)$$

Algorithm 1 GPI [60]

Input: Matrices $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times d}$, tolerance $\varepsilon > 0$.
Output: $P = \arg \min_{P^T P = I} Tr(P^T A P) - 2Tr(P^T B)$.

- 1 Initialize P such that $P^T P = I$.
- 2 Let $H \leftarrow (1 + \sigma_{\max}(A))I - A$.
- 3 **while** $t < \text{MAX_ITER}$ and not convergent **do**
- 4 Compute $[\Pi, \Sigma, \Gamma] \leftarrow \text{SVD}(HP + B, \text{"econ"})$.
- 5 **if** $\|P - \Pi \Gamma\| < \varepsilon$ **then**
- 6 **break**;
- 7 **end if**
- 8 Let $P \leftarrow \Pi \Gamma$.
- 9 Let $t \leftarrow t + 1$.
- 10 **end while**

where λ_l denotes the Lagrange multiplier and is obtained by the following dual problem:

$$\min_{0 \leq \lambda_l \leq 1} \frac{1}{2} \lambda_l^T Q_{K,K}^{(l)} \lambda_l + \left(Q_{\Psi,K}^{(l)} \hat{P} v_l - \mathbf{1} \right)^T \lambda_l. \quad (20)$$

We leave the detailed deduction in Appendix A in the supplementary material.

V Step: Fixing $\hat{U}$ and $\hat{P}$, we have

$$\min_{v_l} \frac{r_1}{2} \|\hat{u}_l - \hat{P} v_l\|^2 + \frac{r_2}{2} \|v_l\|^2.$$

Concisely, setting the derivative of v_l to zero, we obtain

$$v_l = \frac{r_1}{r_1 + r_2} \hat{P}^T u_l. \quad (21)$$

P Step: We optimize P by the following problem:

$$\min_{\hat{P}^T \hat{P} = I} r_1 \sum_{l=1}^K \tau_l \|\hat{u}_l - \hat{P} v_l\|^2 + \mu Tr(\hat{P}^T \Psi L \Psi^T \hat{P}). \quad (22)$$

With some simple algebra, the problem becomes

$$\min_{\hat{P}^T \hat{P} = I} \mu Tr(\hat{P}^T \Psi^T L \Psi \hat{P}) - 2r_1 Tr(\hat{P}^T \hat{U} T V^T) \quad (23)$$

where $T = \text{diag}(\tau)$. Problem (23) is a QPP over Stiefel manifold $\text{St}(\mathbb{R}^n, d)$. Here, we leverage an efficient solver, i.e., generalized power iteration (GPI) [60]. Note that (23) is equivalent to

$$\max_{\hat{P}^T \hat{P} = I} Tr(\hat{P}^T H \hat{P}) + 2Tr(\hat{P}^T E) \quad (24)$$

where $E = r_1 \hat{U} T V^T$ and $H = \sigma I - \mu \Psi^T L \Psi$ with a $\sigma > 0$ such that H is positive-definite. The large σ might cause slow convergence [60]. Thus, we can set $\sigma = 1 + \mu \sigma_{\max}(\Psi^T L \Psi)$, where $\sigma_{\max}(\cdot)$ gives the largest singular value of the input matrix. GPI generates a series of $\hat{P}_t$ converging to a local minimizer of (23) via iteration: $\hat{P}_{t+1} = \pi_{\text{St}}(H \hat{P}_t + E)$. Specifically, the Euclidean projection of A onto Stiefel manifold is $\pi_{\text{St}}(A) = \Pi \Pi^T$ with the compact SVD: $\Pi \Sigma \Pi^T = A$ [61]. We summarize the GPI procedure in Algorithm 1.

τ Step: In this step, we solve the maximization problem (15) to update τ . We first consider the equality constraint $\nabla_{\text{sh}} \tau = \mathbf{0}$ in (15) with the shared variable $\theta_{\text{sh}} = \hat{P}$. Note that P is constrained in the Stiefel manifold $\text{St}(\mathbb{R}^n, d)$. Therefore, we should utilize the corresponding Riemannian gradient instead

Algorithm 2 Training K-NPSVC++

Input: Samples $X \in \mathbb{R}^{m \times n}$, $y \in [K]^n$.
Output: Matrix $\hat{U} \in \mathbb{R}^{n \times K}$, $V \in \mathbb{R}^{d \times K}$, and projection matrix $\hat{P} \in \mathbb{R}^{n \times d}$.

- 1 Initialize $\hat{U}_0, V_0, \hat{P}_0$ such that $\hat{P}_0^T \hat{P}_0 = I$, and $\tau_0 = \mathbf{1}/K$.
- 2 Let $t \leftarrow 0$.
- 3 Take Ψ such that $\Psi \Psi^T = K$, where K is kernel matrix.
- 4 Compute $S \leftarrow \mu \Psi^T L \Psi$.
- 5 **while** $t < \text{MAX_ITER}$ and not convergent **do**
- 6 **for** $l \in [K]$ **do**
- 7 Solve the problem (20).
- 8 Update $\hat{u}_{t+1,l} \leftarrow r_1 Q_{\Psi,\Psi}^{(l)} \hat{P}_t v_{t,l} + Q_{K,\Psi}^{(l)} \lambda_l$.
- 9 **end for**
- 10 Update $V_{t+1} \leftarrow r_1 \hat{P}_t^T \hat{U}_{t+1} / (r_1 + r_2)$.
- 11 Update $E \leftarrow r_1 \hat{U}_{t+1} \text{diag}(\tau_t) V_{t+1}^T$.
- 12 Update $\hat{P}_{t+\frac{1}{2}} \leftarrow \text{GPI}(S, E, P_t)$ with Algorithm 1.
- 13 Update τ_{t+1} by solving QPP (25).
- 14 Let $\hat{P}_{t+1} \leftarrow \pi_{\text{St}}\left(\hat{P}_{t+\frac{1}{2}} - \eta \sum_{l=1}^K \tau_{t+1,l} \nabla_{\hat{P}_{t+\frac{1}{2}}}^{\text{St}} J_l\right)$.
- 15 Let $t \leftarrow t + 1$.
- 16 **end while**

of the gradient in Euclidean space. According to [62], the Riemannian gradient over the Stiefel manifold is

$$\nabla_{\hat{P}}^{\text{St}} J_l = \left(\nabla_{\hat{P}} J_l \hat{P}^T - \hat{P} \nabla_{\hat{P}} J_l^T \right) \hat{P}$$

where $\nabla_{\hat{P}} J_l = \mu \Psi^T L \Psi \hat{P} - r_1 u_l v_l^T$. Therefore, we optimize the relaxed problem of (15) instead

$$\min_{\tau \in \Delta} \frac{1}{2} \left\| \sum_{l=1}^K \tau_l \nabla_{\hat{P}}^{\text{St}} J_l \right\|^2 - \gamma \sum_{l=1}^K \tau_l J_l \quad (25)$$

where $\gamma > 0$ is a hyperparameter. This is a QPP with a probability simplex constraint. An efficient solver is the double-coordinate descent method [63]. Remarkably, the following theorem demonstrates a valuable property of the optimal τ .

Theorem 2 ([55], [57]): If $\gamma = 0$, let τ^* be the optimal solution to (25), and we have the following.

- 1) If the minimum is zero, then $\hat{P}$ is a Pareto stationary point.
- 2) Otherwise, $\Delta P = -\sum_{l=1}^K \tau_l^* \nabla_{\hat{P}}^{\text{St}} J_l$ is the direction that descends all J_l , $\forall l \in [K]$.

In a nutshell, the optimal solution to (25) yields a direction that reduces all objectives when $\gamma = 0$, which is a by-product of updating τ . Therefore, we can utilize the projected gradient descent step to further update $\hat{P}$

$$\hat{P} := \pi_{\text{St}}\left(\hat{P} - \eta \sum_{l=1}^K \tau_l^* \nabla_{\hat{P}}^{\text{St}} J_l\right)$$

where $\eta > 0$ is the learning rate. In practice, we should set $\gamma > 0$, and thus, the presence of the dual maximization term in (25) will yield a perturbation in this descending direction. Hence, we should determine a proper γ to balance the two terms in (25) and thereby achieve a tradeoff between Pareto stationarity and dual maximization. The training of

Algorithm 3 Training D-NPSVC++

```

1 for  $p = 1 : n_{epoch}$  do
2   for each mini-batch  $\mathcal{B} = \{\mathbf{x}_{i_1}, \dots, \mathbf{x}_{i_{|\mathcal{B}|}}\}$  do
3     Activate the parameter  $\tilde{\mathbf{W}}$ .
4     # First-step (14).
5     Cache the representations from prior encoder  $\Phi_{\mathcal{B}} \leftarrow [\phi(\mathbf{x}_{i_1}), \dots, \phi(\mathbf{x}_{i_{|\mathcal{B}|}})]$ .
6     Compute  $\mathbf{Z}_{\mathcal{B}} \leftarrow [z(\phi(\mathbf{x}_{i_1})), \dots, z(\phi(\mathbf{x}_{i_{|\mathcal{B}|}}))]$ , and let  $\tilde{\mathbf{Z}}_{\mathcal{B}} = \text{Concat}(\Phi_{\mathcal{B}}, \mathbf{Z}_{\mathcal{B}})$ .
7     Compute  $\hat{f}_{il} \leftarrow \tilde{\mathbf{w}}_l^T \tilde{\mathbf{z}}_i, \forall i \in [i_1, \dots, i_{|\mathcal{B}|}], l \in [K]$ .
8     Compute the objectives for  $\forall l \in [K]: J_l = \frac{1}{|\mathcal{B}|} \left[ \sum_{y_l=l} \ell_s(\hat{f}_{il}) + \frac{c}{K-1} \sum_{y_l \neq l} \ell_d(\hat{f}_{il}) \right]$ .
9     Compute  $J = \sum_{l=1}^K \tau_l J_l$ , and apply gradient back-propagation and update parameters  $[\tilde{\mathbf{W}}, \theta_{sh}]$ .
10    # Second-step (15).
11    Freeze the parameter  $\tilde{\mathbf{W}}$ .
12    Compute  $\mathbf{Z}_{\mathcal{B}} \leftarrow [z(\phi(\mathbf{x}_{i_1})), \dots, z(\phi(\mathbf{x}_{i_{|\mathcal{B}|}}))]$ , and let  $\tilde{\mathbf{Z}}_{\mathcal{B}} = \text{Concat}(\Phi_{\mathcal{B}}, \mathbf{Z}_{\mathcal{B}})$ .
13    Again compute the objectives  $J_l$  and the gradient  $\nabla_{\mathbf{Z}_{\mathcal{B}}} J_l$  for  $\forall l \in [K]$ .
14    Solve the QPP (30) and update  $\tau$  with (31).
15  end for
16 end for

```

K-NPSVC++ is summarized in Algorithm 2. In the testing phase, the following prediction function is used:

$$\begin{aligned}
h(\mathbf{x}') &= \arg \min_{l \in [K]} \frac{|f_l(\mathbf{x}')|}{\sqrt{\|\mathbf{w}_l\|^2 + \|\mathbf{v}_l\|^2}} \\
&= \arg \min_{l \in [K]} \frac{|f_l(\mathbf{x}')|}{\sqrt{\|\hat{\mathbf{u}}_l - \hat{\mathbf{P}}\mathbf{v}_l\|^2 + \|\mathbf{v}_l\|^2}}
\end{aligned} \quad (26)$$

where $f_l(\mathbf{x}) = \sum_{i=1}^n \beta_{il} \mathcal{K}(\mathbf{x}_i, \mathbf{x})$ and $\beta_l = \Psi^{-T} \hat{\mathbf{u}}_l$. In particular, if the linear kernel is used, one can directly compute $f_l(\mathbf{x}) = \hat{\mathbf{u}}_l^T \mathbf{x}$ without accessing β_l .

The scalability of K-NPSVC++ is relatively limited, which is a common issue in kernel methods. Thus, it is more suitable for small datasets, and we recommend handling large-scale data using the subsequent D-NPSVC++.

B. NPSVC++ in Deep Learning: D-NPSVC++

We introduce D-NPSVC++, the deep version of NPSVC++. First, note that (10) is formulated as

$$f_l(\mathbf{x}) = \begin{bmatrix} \mathbf{w}_l \\ \mathbf{v}_l \end{bmatrix}^T \begin{bmatrix} \phi(\mathbf{x}) \\ \mathbf{z}(\mathbf{x}) \end{bmatrix} = \tilde{\mathbf{w}}_l^T \tilde{\mathbf{z}}(\mathbf{x}) \quad (27)$$

which can be constructed as a linear layer. The prior encoder $\phi(\cdot)$ could be a pretrained network (e.g., ResNet34 in our experiments). For simplicity, we parameterize the encoder with a multilayer perceptron (MLP): $\mathbf{z}(\mathbf{x}) = \text{MLP}(\phi(\mathbf{x}))$.

The training of D-NPSVC++ still follows the criterion (13). In the first step (14), we optimize the network parameters with the backpropagation algorithm, following the criterion:

$\min_{\theta} \sum_{l=1}^K \tau_l J_l$. Similar to K-NPSVC++, the second step (15) solves the following problem:

$$\min_{\tau \in \Delta} \frac{1}{2} \left\| \sum_{l=1}^K \tau_l \nabla_{\theta_{sh}} J_l \right\|^2 - \gamma \sum_{l=1}^K \tau_l J_l. \quad (28)$$

However, the shared parameter θ_{sh} [i.e., the parameter of $\mathbf{z}(\cdot)$] is generally quite high dimensional. Therefore, it is of high possibility that $\mathbf{0} \notin \text{span}(\{\nabla_{\theta_{sh}} J_l\}_{l=1}^K)$, which makes it difficult to minimize the first term of (28). Inspired by [53], we can first leverage the chain rule and the property of the norm and obtain

$$\left\| \sum_{l=1}^K \tau_l \nabla_{\theta_{sh}} J_l \right\| \leq \left\| \frac{\partial \mathbf{Z}_{\mathcal{B}}}{\partial \theta_{sh}} \right\| \left\| \sum_{l=1}^K \tau_l \nabla_{\mathbf{Z}_{\mathcal{B}}} J_l \right\| \quad (29)$$

where $\mathbf{Z}_{\mathcal{B}} = [\mathbf{z}(\mathbf{x}_i)]_{\mathbf{x}_i \in \mathcal{B}}$ is the representations of the mini-batch $\mathcal{B}$. Note that the dimension of $\mathbf{Z}_{\mathcal{B}}$ is significantly lower than that of θ_{sh} , and the left-hand-side $\|\sum_{l=1}^K \tau_l \nabla_{\theta_{sh}} J_l\| \rightarrow 0$ if $\|\sum_{l=1}^K \tau_l \nabla_{\mathbf{Z}_{\mathcal{B}}} J_l\| \rightarrow 0$. Hence, we indirectly optimize (28) by minimizing its upper bound

$$\tau^* = \arg \min_{\tau \in \Delta} \frac{1}{2} \left\| \sum_{l=1}^K \tau_l \nabla_{\mathbf{Z}_{\mathcal{B}}} J_l \right\|^2 - \gamma \sum_{l=1}^K \tau_l J_l. \quad (30)$$

Additionally, the optimal solution τ^* is probably sparse. The sparse τ^* potentially hinders the gradient propagation for the objectives corresponding to the zero entries, and further causes slow convergence. To alleviate this problem, we adopt a momentum strategy to update τ

$$\tau_{t+1} = \beta \tau^* + (1 - \beta) \tau_t \quad (31)$$

where $0 < \beta < 1$ is a momentum hyperparameter. This still guarantees the monotonic decrease of the objective in (30). To see this, let the objective function be $g(\tau)$. We can know that $g(\tau)$ is σ^2 -strongly convex and σ is the minimal singular value of $\nabla_{\mathbf{Z}_{\mathcal{B}}} \mathbf{J}$, and

$$\begin{aligned}
g(\tau_{t+1}) &\leq \beta g(\tau^*) + (1 - \beta) g(\tau_t) - \frac{\sigma^2}{2} \beta (1 - \beta) \|\tau_t - \tau^*\|^2 \\
&= g(\tau_t) + \beta (g(\tau^*) - g(\tau_t)) \\
&\quad - \frac{\sigma^2}{2} \beta (1 - \beta) \|\tau_t - \tau^*\|^2 \\
&\leq g(\tau_t) - \frac{\sigma^2}{2} \beta (1 - \beta) \|\tau_t - \tau^*\|^2 \leq g(\tau_t).
\end{aligned}$$

The first inequality follows the property of the strongly convex function, and the second inequality is based on the fact that τ^* minimizes $g(\cdot)$. We set $\beta = 0.4$ in the experiments. The details of training D-NPSVC++ are stated in Algorithm 3.

V. CONVERGENCE AND COMPLEXITY

We give the convergence analysis of NPSVC++ as follows.

Theorem 3 (Informal): Let $\{(\theta^t, \tau^t)\}$ be the sequence generated by NPSVC++ updates. Assume that J_l has uniformly bounded Hessian with respect to θ_{sh} , for a sufficiently small learning rate $\eta > 0$ and large enough $\gamma > 0$, the sequence $\{(\theta^t, \tau^t)\}$ converges to a Pareto stationary point.

The formal statement and proof are provided in the Appendix in the supplementary material. Although

TABLE II
STATISTICAL INFORMATION OF DATASETS USED IN THE EXPERIMENTS

Dataset	#Train	#Test	#Dimension	#Class	Source
CHG	540	360	8000	10	[64]
W3a	2947	1965	300	2	[31]
BinAlpha	842	562	320	36	Website ²
20News	9745	6497	100	4	[65]
Pendigits	6595	4397	16	10	UCI
DNA	1912	1274	180	3	StatLog
USPS	4374	2917	256	10	[66]
DTD	1880	1880	4096	47	[67]
FLW-102	8189	2048	1024	102	[68]
Cifar-10	50000	10000	3072	10	[69]
F-MNIST	60000	10000	784	10	[70]
SVHN	73257	26032	3072	10	[71]

our result needs a strong assumption on uniformly bounded Hessian, it is satisfied in K-NPSVC++ (22) and D-NPSVC++ with squared hinge loss. Theorem 3 guarantees the convergence to a Pareto stationary point in NPSVC++, providing a strong theoretical support for its effectiveness.

We analyze the computational complexity of K-NPSVC++. Let ϵ indicate the tolerance of error. The space complexity is $O(n^2)$ to store Ψ . Cholesky decomposition is computed once before iterations, costing $O(n^3)$ time. In U step, the QPPs are solved by the SOR algorithm with time complexity of $O(n^2 K \log(1/\epsilon))$ by Mangasarian and Musicant [72]. V step needs $O(nK)$ time. The GPI procedure in P step requires $O(T_{\text{gpi}} n d^2)$, where T_{gpi} is the number of GPI iterations. Finally, the time of τ step is $O(k^2 \log(1/\epsilon))$ by Chen et al. [73]. Regardless the log-scale constants, the total time is roughly $O(n^3 + T_{\text{out}}(n^2 K + T_{\text{gpi}} n d^2 + k^2))$ time, where T_{out} is the iteration times of outer loop, empirically $T_{\text{out}} < 10$.

Compared to DNNs without tuning the weights τ , D-NPSVC++ solves an extra QPP (30) and has $O(K^2 \log(1/\epsilon))$ additional time consumption. Nevertheless, since K is generally a relatively small value, this cost is acceptable in most cases.

VI. EXPERIMENTS

In the experiments, we first compared the proposed K-NPSVC++ with both the generally used and the latest NPSVCs. We emphasize that although NPSVC++ is derived using multiobjective principles, the focus of this article is on their effectiveness as nonparallel classifiers. Therefore, in this work, we compare NPSVC++ against the mainstream classifiers, particularly SVM-based and recent NPSVC variants, rather than with multitask learning or multiobjective optimization methods. Next, we evaluate D-NPSVC++ in comparison to the DNNs equipped with the learning criteria. For fairness, the same network architecture is used for these methods. Finally, the behaviors of both K-NPSVC++ and D-NPSVC++ are studied empirically. In our experiments, we use Gaussian kernel $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2/t)$ and t is empirically set as $t = \sum_{i,j} \|\mathbf{x}_i - \mathbf{x}_j\|^2 / (n(n-1))$. This kernel setting is applied to all compared methods. The graph G is constructed as a k nearest neighbor graph following [40], i.e.,

$$G_{ij} = \begin{cases} \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j), & i \in \mathcal{N}_k(j) \vee j \in \mathcal{N}_k(i) \\ 0, & \text{Otherwise} \end{cases} \quad (32)$$

where $\mathcal{K}(\cdot, \cdot)$ is the Gaussian kernel defined above, and $\mathcal{N}_k(i)$ denotes the index set of the k nearest neighborhood of $\mathbf{x}_i$. The hyperparameter k , for convenience, is set as $k = \lfloor \log_2 n \rfloor$, where n is the size of the training set. We use the normalized Laplacian $L = I - D^{-1/2} G D^{-1/2}$ for better computational stability. Twelve datasets are used in the experiments, where their basic information is shown in Table II. The optimal c, r_1 , and r_2 are tuned in $\{10^i : i = -5, -4, \dots, 5\}$, and μ is tuned around K/n inspired by [58].

The implementation of D-NPSVC++ is based on PyTorch.³

In our experiments, we apply the square loss as similarity loss and the squared-hinge loss as dissimilarity loss, i.e.,

$$\ell_s(f_i(\mathbf{x}_i)) = \frac{1}{2} [f_i(\mathbf{x}_i)]^2, \quad \ell_d(f_i(\mathbf{x}_i)) = \frac{1}{2} [s - f_i(\mathbf{x}_i)]_+^2.$$

Different from K-NPSVC++, whose dissimilarity loss is a nondifferentiable function, $\ell_d(f_i(\mathbf{x}_i)) = [s - f_i(\mathbf{x}_i)]_+$, we choose the squared-hinge loss in D-NPSVC++ for its differentiability. The margin size s is selected from $\{2^i : i = 0, 1, \dots, 6\}$. We train the DNNs for 50 epochs and use a batch size of 128. The model is trained with stochastic gradient descent (i.e., SGD) with a momentum parameter 0.9. The weight decay is set as 10^{-4} . The initial learning rate is set as 10^{-2} for FLW dataset and 10^{-3} for other datasets. We use the cosine annealing strategy proposed by Loshchilov and Hutter [74] to schedule the learning rate, with maximal iteration of 50 and minimal learning rate of 10^{-5} . These settings are also applied to the compared methods.⁴

A. Comparison of SVMs

We compared K-NPSVC++ with the existing SVMs, particularly NPSVCs, including vanilla SVM [1] and representative NPSVCs, including TWSVM [17], NPSVM [13], pinTWSVM [18], and RMTBSVM [36]. We also compare NPSVC++ with two methods that learn features and a classifier separately (we call them two-stage methods). GBTSVM leverages granular balls to generate robust features [49], and FTWSVM-CKA [48] selects the features using information gain and kernel alignment. The datasets include the CHG dataset for hand gesture recognition [64], BinAlpha for hand-written recognition, and datasets available at the LIBSVM website.⁵ The average accuracy of ten runs with random split, where the 60% samples of the whole dataset are used for training and the rest for testing, is reported in Table III. It can be seen that K-NPSVC++ always outperforms the other compared methods. Moreover, we conducted paired t-tests to evaluate the statistical significance following [34]. In Table III, we use the notation “ $\ddagger$ ” (or “ $\dagger$ ”) to denote the statistically significant result $p < 0.05$ (or $p < 0.1$). It can be observed that the performance of K-NPSVC++ usually surpasses the other methods with a significance level of 0.05. These results demonstrate that the features learned by K-NPSVC++ improve the classification performance of NPSVC. Specifically, the existing methods directly use the nonparametric features

³<https://pytorch.org/>

⁴We use a subset (with 20 000 samples) of the training set to train NPSVCs on Cifar10, FMNIST, and SVHN datasets to avoid memory overflow.

⁵<https://www.csie.ntu.edu.tw/~cjlin/libsvmtools/datasets/>

TABLE III

CLASSIFICATION PERFORMANCE OF SVMs. “ $\frac{+}{-}$ ” AND “ $\frac{+}{+}$ ” DENOTE THE SIGNIFICANCE LEVEL OF 0.05 AND 0.1, RESPECTIVELY. THE TOP-1 AND TOP-2 RESULTS ARE BOLD AND UNDERLINED, RESPECTIVELY

Dataset	Methods							K-NPSVC++
	SVM [1]	TWSVM [11]	NPSVM [33]	pinTWSVM [18]	RMTBSVM [36]	GBTSVM [49]	FTWSVM-CKA [48]	
CHG	82.08 $\pm$ 2.01 $\frac{+}{-}$	85.75 $\pm$ 2.41 $\frac{+}{-}$	86.83 $\pm$ 3.14 $\frac{+}{-}$	<u>87.42</u> $\pm$ 2.61 $\frac{+}{+}$	87.28 $\pm$ 2.44 $\frac{+}{-}$	85.08 $\pm$ 1.97 $\frac{+}{-}$	86.58 $\pm$ 2.75 $\frac{+}{-}$	88.14 $\pm$ 2.21
W3a	97.02 $\pm$ 0.41 $\frac{+}{-}$	97.89 $\pm$ 0.32 $\frac{+}{-}$	97.76 $\pm$ 0.33 $\frac{+}{-}$	97.36 $\pm$ 0.39 $\frac{+}{-}$	97.96 $\pm$ 0.30 $\frac{+}{-}$	97.31 $\pm$ 0.42 $\frac{+}{-}$	98.07 $\pm$ 0.30	<u>98.05</u> $\pm$ 0.28
BinAlpha	<u>71.10</u> $\pm$ 2.19	68.06 $\pm$ 1.86 $\frac{+}{-}$	69.59 $\pm$ 1.63 $\frac{+}{-}$	64.50 $\pm$ 1.46 $\frac{+}{-}$	69.68 $\pm$ 2.42 $\frac{+}{-}$	67.51 $\pm$ 1.45 $\frac{+}{-}$	66.85 $\pm$ 1.77 $\frac{+}{-}$	71.28 $\pm$ 1.79
20News	79.31 $\pm$ 0.43 $\frac{+}{-}$	80.33 $\pm$ 0.37 $\frac{+}{-}$	<u>81.86</u> $\pm$ 0.42	80.11 $\pm$ 0.32 $\frac{+}{-}$	80.72 $\pm$ 0.44 $\frac{+}{-}$	80.04 $\pm$ 0.52 $\frac{+}{-}$	81.67 $\pm$ 0.31 $\frac{+}{-}$	81.92 $\pm$ 0.36
Pendigits	91.75 $\pm$ 0.26 $\frac{+}{-}$	99.35 $\pm$ 0.10	<u>99.38</u> $\pm$ 0.09	95.45 $\pm$ 0.48 $\frac{+}{-}$	99.04 $\pm$ 0.16 $\frac{+}{-}$	98.94 $\pm$ 0.16 $\frac{+}{-}$	99.26 $\pm$ 0.12 $\frac{+}{-}$	99.39 $\pm$ 0.08
DNA	95.01 $\pm$ 0.52 $\frac{+}{-}$	95.94 $\pm$ 0.35 $\frac{+}{-}$	95.68 $\pm$ 0.55 $\frac{+}{-}$	95.45 $\pm$ 0.68 $\frac{+}{-}$	96.10 $\pm$ 0.60	93.45 $\pm$ 1.16 $\frac{+}{-}$	<u>96.16</u> $\pm$ 0.54	96.19 $\pm$ 0.54
USPS	96.63 $\pm$ 0.30 $\frac{+}{-}$	97.51 $\pm$ 0.32 $\frac{+}{-}$	97.14 $\pm$ 0.39 $\frac{+}{-}$	<u>98.06</u> $\pm$ 0.26	97.86 $\pm$ 0.22 $\frac{+}{-}$	93.33 $\pm$ 0.55 $\frac{+}{-}$	97.45 $\pm$ 0.35 $\frac{+}{-}$	98.08 $\pm$ 0.24

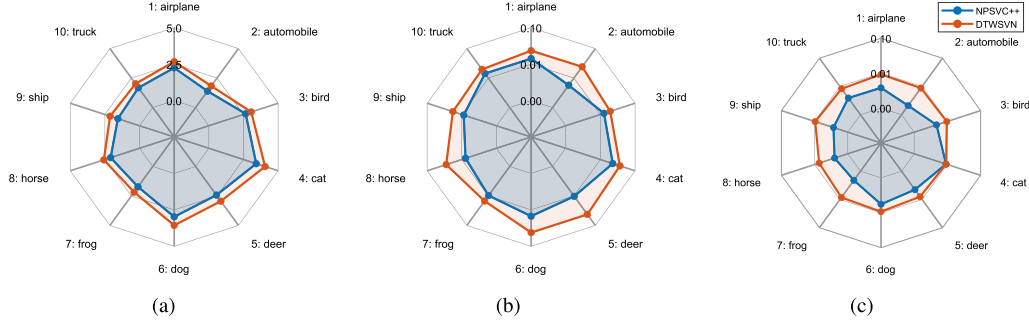


Fig. 4. Radar chart of the objectives of D-NPSVC++ and DTWSVN on Cifar-10. We plot the objective of each class in the first, the 25th, and the last epochs. (a) First epoch. (b) 25th epoch (log-scaled axes). (c) Final epoch (log-scaled axes).

TABLE IV

CLASSIFICATION PERFORMANCE OF DIFFERENT DEEP MODELS AND KERNEL NPSVCs. NPSVCs USE THE FEATURES OF THE PRETRAINED RESNET34. WE HIGHLIGHT THE BEST AND THE SECOND BEST RESULTS

Type	Criterion	Datasets				
		Cifar-10	DTD	FLW	FMNIST	SVHN
DNN	Cross-entropy	95.19	64.47	81.67	93.96	95.89
	ArcFace [44]	95.33	64.47	82.45	94.36	96.31
	DTWSVN [29]	95.30	65.11	<u>84.44</u>	93.67	<u>96.73</u>
	DeepSVM [27]	95.48	65.00	83.83	<u>94.06</u>	96.11
	D-NPSVC++	96.77	<u>65.90</u>	86.27	94.68	97.33
NPSVC ⁴	TWSVM	89.83	64.94	77.70	88.23	61.01
	NPSVM	89.16	64.52	78.58	88.11	61.57
	RMTBSVM	89.74	65.05	74.40	88.51	61.09
	FTWSVM-CKA	89.24	64.18	75.98	88.42	58.56
	K-NPSVC++	91.13	66.54	80.63	89.15	61.89

$\phi(x)$ induced by the kernel function for classification, while K-NPSVC++’s features $z(x)$ are optimized and maximize the performance of NPSVC. Besides, K-NPSVC++ significantly outperforms FTWSVM-CKA and GBTSVM on five out of seven datasets, which suggests that the feature learning strategy of NPSVC++ is better than the two-stage way in FTWSVM-CKA and GBTSVM. This is because the features learned by K-NPSVC++ are Pareto optimal, while FTWSVM-CKA and GBTSVM cannot guarantee the optimality of the obtained features. These results demonstrate the superiority of K-NPSVC++.

B. Real-World Image Classification

D-NPSVC++ was compared to DNNs with the criteria including cross-entropy (i.e., softmax regression), margin softmax regression in [44], and deep TWSVM [29]. Five image

classification datasets are involved, i.e., Cifar-10 [69], DTD [67], and Flowers-102 (FLW for short) [68], F-MNIST [70] and SVHN [71] datasets. We also include the comparison with the kernel machines, TWSVM and K-NPSVC++, with the feature extracted by ResNet34 as their input. As shown in Table IV, D-NPSVC++ has beaten other criteria on three datasets. Compared with deep TWSVM [29], D-NPSVC++ shows a significant improvement in classification performance, which validates the effectiveness of the proposed training criterion of NPSVC++. Interestingly, K-NPSVC++ defeats DNNs and achieves the best performance on the DTD dataset, and it also performs better than other SVMs significantly. The reason may be that K-NPSVC++ found a better feature space in RKHS than DNNs on the DTD dataset. This phenomenon shows that K-NPSVC++, as a kernel machine, still has competitive power against D-NPSVC++. Therefore, we conclude that both K-NPSVC++ and D-NPSVC++ present promising performance, and it is also pivotal to select K-NPSVC++ or D-NPSVC++ for classification according to the expected performance and computation budgets.

C. Property Study of NPSVC++

1) *Objective Visualization*: To show that the proposed D-NPSVC++ achieves a better solution to the multiobjective optimization problem (7) than DTWSVN [29], we plot the radar charts to illustrate the objectives of each class in Fig. 4. Both methods compute the objective of the l th class by

$$J_l = \frac{1}{n} \left[\sum_{y_i=l} \ell_s(f_l(x_i)) + \frac{c}{K-1} \sum_{y_i \neq l} \ell_d(f_l(x_i)) \right] \quad (33)$$

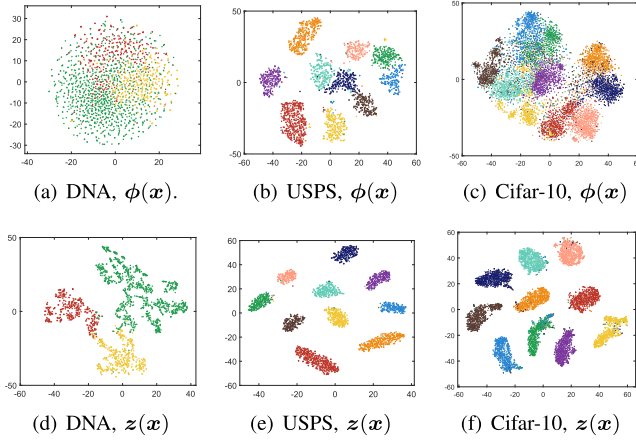


Fig. 5. (a)–(c) t-SNE visualization of raw data and (d)–(f) learned features.

where the two methods use the same loss functions. It can be observed that D-NPSVC++ always achieves lower objectives in all classes than DTWSVN, which suggests the superiority of D-NPSVC++ over DTWSVN and the effectiveness of the proposed training strategy. To explain this phenomenon, we compare the optimization criterion of these two methods

$$\text{DTWSVN} : \min_{\theta} \frac{1}{K} \sum_{l=1}^K J_l \quad (34)$$

$$\text{D-NPSVC++} : \begin{cases} \theta\text{step: } \min_{\theta} \sum_{l=1}^K \tau_l J_l \\ \tau\text{step: } \max_{\tau \in \Delta} \sum_{l=1}^K \tau_l J_l \\ \text{s.t. } \sum_{l=1}^K \tau_l \nabla_{Z_B} J_l = \mathbf{0}. \end{cases} \quad (35)$$

DTWSVN uses uniform scaling methods to minimize multiple dependent objectives and usually fails to achieve the optimal solution [53]. The reason is that it does not consider the relationship between different objectives. In contrast, the proposed D-NPSVC++ aims to achieve Pareto optimality, which implements the tradeoff of different objectives. Thus, D-NPSVC++ can obtain a better solution than DTWSVN.

2) *Learned Features*: The learned features of K-NPSVC++ are illustrated in Fig. 5. It can be observed that the learned features $z(x)$ are more discriminative than the raw features $\phi(x)$. The intraclass distance is squeezed, and the interclass margin is enlarged. From a quantitative perspective, the Dirichlet energy of the learned feature is reduced compared to raw data (e.g., the Dirichlet energy is decreased from 0.7174 to 0.0013 on DNA dataset), indicating substantially improved separability [75]. This suggests that K-NPSVC++ can learn discriminative features and effectively enhance classification performance.

3) *Convergence*: We depict the convergence of K-NPSVC++ in Fig. 6(a). The lines “primal” and “dual” refer to the objective of problem (12) and (13). Intuitively, it can be seen that the primal objective decreases rapidly and merges with the dual objective within a few steps, which indicates optimality since the dual gap becomes zero.

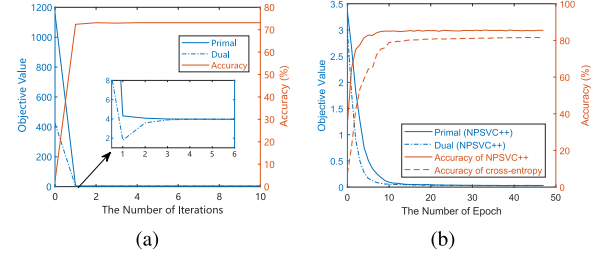


Fig. 6. Training progress of K-NPSVC++ and D-NPSVC++. (a) K-NPSVC++ on BinAlpha. (b) D-NPSVC++ on FLW.

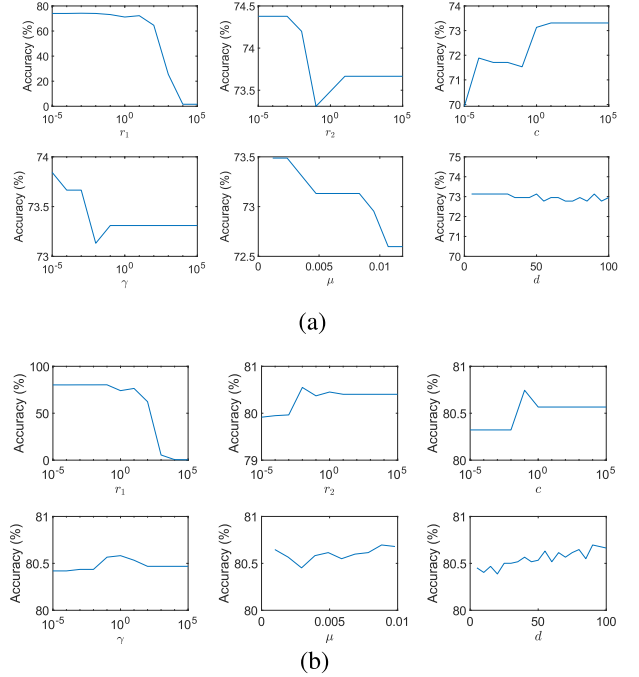


Fig. 7. Hyperparameter of K-NPSVC++. (a) BinAlpha. (b) FLW (feature extracted by ResNet-34).

TABLE V
ACCURACY VERSUS THE MOMENTUM PARAMETER β

Dataset	Momentum parameter β in (31)					
	0.0	0.2	0.4	0.6	0.8	1.0
FLW	84.44	85.44	86.39	85.41	85.17	84.36
Cifar-10	95.30	95.95	96.49	95.82	95.90	95.37

Meanwhile, the accuracy also grows fast and finally becomes stable. Empirically, K-NPSVC++ can reach convergence within five iterations. Fig. 6(b) shows the training evolution of D-NPSVC++. Similar to K-NPSVC++, the primal and dual objectives of D-NPSVC++ decrease monotonically and converge to a minimum. Compared with the cross-entropy, D-NPSVC++ achieves the highest accuracy within fewer epochs, which suggests that D-NPSVC++ could perform faster convergence than softmax regression.

4) *Hyperparameters*: Since K-NPSVC++ involves many hyperparameters, we study its performance versus different hyperparameter settings. The experimental results are shown in Fig. 7. We observed that K-NPSVC++ works stably for small r_1 , but performs poorly when r_1 is extremely large. When r_1

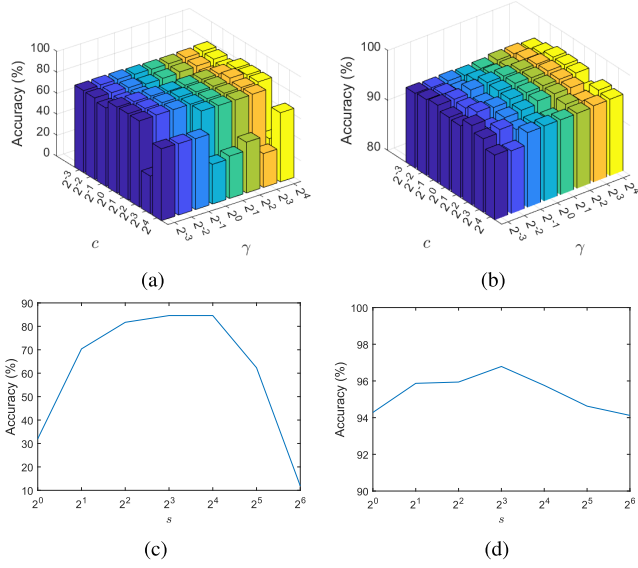


Fig. 8. Hyperparameters of D-NPSVC++ on FLW and Cifar-10. (a) and (c) FLW. (b) and (d) Cifar-10.

is large, the model tends to make $\|w_l\| \rightarrow 0$, or equivalently, $u_l - P v_l \rightarrow 0$, which probably leads to a trivial subspace and results in model degradation. Therefore, K-NPSVC++ is sensitive to r_1 and a small r_1 is preferred. On the other hand, it is relatively insensitive to other hyperparameters (r_2, c, μ, γ). We also found that the optimal d may be close to the number of classes K . In practice, we can first choose a small r_1 and $d \approx K$ and tune other parameters via grid search or Bayesian optimization. In D-NPSVC++, the tunable hyperparameters include c, γ , and the margin size s . Theorem 3 suggested that the large γ ensures the convergence of D-NPSVC++, while too large γ may weaken the Pareto stationarity term in (30). Thus, a moderate γ is required to balance convergence and performance during optimization. As shown in Fig. 8, we can find that $c \in [1, 4]$ and $\gamma \in [1, 4]$ usually deliver a satisfactory and stable performance. In addition, either too large or too small s will degrade the accuracy, and the optimal range is $s \in [4, 16]$. We further note that the optimal range of s is typically independent of c and γ . Therefore, we can first determine the optimal s before tuning c and γ . Finally, we investigate the performance versus the momentum parameter β , as shown in Table V. Empirically, $\beta = 0.4$ is moderate and usually gives satisfying performance. $\beta = 0$ indicates that τ is never updated, and the performance is equivalent to DTWSVN, which reduces all objectives uniformly and is not optimal. On the other hand, $\beta = 1$ means always using τ^* in (30), which causes suboptimal performance because its sparsity vanishes the gradient of objectives associated with the zero entries and may hinder their optimization. In this sense, β plays a role in balancing the optimality of the Lagrangian multipliers τ and the optimality of primal objectives $J_l, l \in [K]$.

5) *Hypothesis Function*: We conducted an ablation study to examine the rationality of the proposed hypothesis (10), which is compared with D-NPSVC++ trained with $f_l(x) = v_l^\top z(x)$ and $f_l(x) = w_l^\top \phi(x)$, i.e., taking only one part in

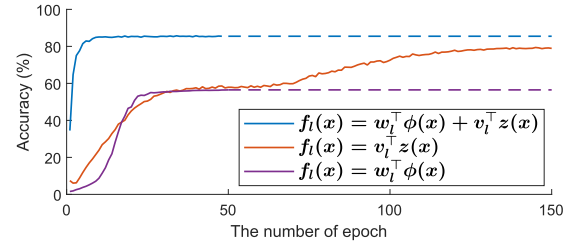


Fig. 9. Ablation study of D-NPSVC++ in hypothesis function. Dashed lines indicate the convergence accuracy.

TABLE VI

TRAINING TIME OF DIFFERENT DNNs (THE FIRST FOUR ROWS) AND KERNEL MACHINES (THE LAST TWO ROWS). THE HARDWARE IS CPU: INTEL XEON SILVER 2.10 GHZ AND GPU: NVIDIA RTX 4090

Model type	Criterion	Datasets		
		Cifar-10	DTD	FLW
DNNs (GPU)	Cross-entropy	49min	39min	48min
	ArcFace [44]	49min	37min	52min
	DTWSVN [29]	34min	20min	33min
	D-NPSVC++ (ours)	42min	22min	42min
Kernel models (CPU)	TWSVM	33min	21.69s	12.67s
	K-NPSVC++ (ours)	56min	34.31s	20.24s

(10) as the compared hypothesis. As shown in Fig. 9, the proposed hypothesis significantly outperforms the two compared hypotheses. Intuitively, $f_l(x) = w_l^\top \phi(x)$ performs the worst as it only depends on the features of the prior encoder. In contrast, $f_l(x) = v_l^\top z(x)$ (w/o skip connection) achieves a better accuracy yet converges slowly. Its reason is that D-NPSVC++ should update τ by minimizing $\|\sum_{l=1}^K \tau_l \nabla_{z_B} J_l\|$, so the gradient is shrunk and the training is slowed consequently. Fortunately, the skip connection structure of the proposed hypothesis $f_l(x) = w_l^\top \phi(x) + v_l^\top z(x)$ can effectively avoid this phenomenon. These results sufficiently validate the efficacy of the designed hypothesis function.

6) *Time Consumption*: We investigate the training time of the proposed NPSVC++. As reported in Table VI, DTWSVN and D-NPSVC++ usually cost less time than the softmax regression methods (cross-entropy and ArcFace). D-NPSVC++ takes slightly more time than DTWSVN since it involves additional computation in solving (30) compared to DTWSVN. It can be observed that K-NPSVC++ and TWSVM have significantly lower time than DNNs on DTD and FLW-102 (small datasets), but they are still time-consuming on Cifar-10 (relatively large). K-NPSVC++ also requires more time to optimize P and τ than the baseline TWSVM method. It shows that NPSVC++ can obtain better features and performance, but at the cost of training time compared to the baselines. Moreover, we can observe (from Table VI) that K-NPSVC++ is significantly faster than D-NPSVC++ when $n < 10000$ (DTD and FLW), but it becomes inefficient when $n > 10000$ (Cifar-10). The main reason is that K-NPSVC++ requires $O(n^3)$ operations to handle the kernel matrix, which stems from the scalability issue of kernel methods. Therefore, K-NPSVC++ is recommended on small-scale datasets, while D-NPSVC++ is suitable for larger datasets. It is worth studying more efficient implementations of NPSVC++ in the future.

VII. CONCLUSION AND FUTURE DIRECTIONS

NPSVCs are powerful yet underutilized classifiers in machine learning. They were born with distinctive designs compared to the mainstream classifiers. Although many works have been proposed, the inherent mechanism of NPSVC has not been well-studied so far. In this article, we establish a new classification framework, NPSVC++, which is the first end-to-end method integrating representation learning and the nonparallel classifier based on multiobjective optimization. NPSVC++ aims to achieve Pareto optimality of multiple dependent objectives, and it theoretically ensures the optimality of the learned features across different classes, solving the issues of feature suboptimality and class dependence naturally. We propose the iterative optimization framework with convergence guarantee and its two instances, K-NPSVC++ and D-NPSVC++, for small- and large-scale datasets, respectively. Their effectiveness is verified in the extensive experiments.

We believe that NPSVC++ paves a new path and provides insights to study and understand NPSVC methods. Some worthy future directions are listed as follows. Diverse multiobjective optimization algorithms could also be the future work. The generalization analysis of NPSVC++ are also appealing directions, which are beyond the scope of this article and require thorough investigations with a new theory. Regarding the influence on the other domain, it is promising to explore the generative model based on NPSVC++ following the Karush–Kuhn–Tucker (KKT) theory established in [45], where Pareto optimality may become a key ingredient.

REFERENCES

- [1] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.
- [2] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*. Cambridge, MA, USA: MIT Press, 2018.
- [3] M. Goudjil, M. Koudil, M. Bedda, and N. Ghogkali, "A novel active learning method using SVM for text classification," *Int. J. Autom. Comput.*, vol. 15, no. 3, pp. 290–298, Jun. 2018.
- [4] M. A. Chandra and S. S. Bedi, "Survey on SVM and their application in image classification," *Int. J. Inf. Technol.*, vol. 13, no. 5, pp. 1–11, Oct. 2021.
- [5] J. A. K. Suykens and J. Vandewalle, "Least squares support vector machine classifiers," *Neural Process. Lett.*, vol. 9, no. 3, pp. 293–300, Jun. 1999.
- [6] R. R. Majeed and S. K. D. Alkhafaji, "ECG classification system based on multi-domain features approach coupled with least square support vector machine (LS-SVM)," *Comput. Methods Biomechanics Biomed. Eng.*, vol. 26, no. 5, pp. 540–547, Apr. 2023.
- [7] T. N. Chan, Z. Li, U. L. Hou, and R. Cheng, "PLAME: Piecewise-linear approximate measure for additive kernel SVM," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 10, pp. 9985–9997, Oct. 2023.
- [8] H. Wang and Y. Shao, "Fast generalized ramp loss support vector machine for pattern classification," *Pattern Recognit.*, vol. 146, Feb. 2024, Art. no. 109987.
- [9] H. Wang, W. Zhou, and Y. Shao, "A new fast ADMM for kernel-less SVM classifier with truncated fraction loss," *Knowl.-Based Syst.*, vol. 283, Jan. 2024, Art. no. 111214.
- [10] O. L. Mangasarian and E. W. Wild, "Multisurface proximal support vector machine classification via generalized eigenvalues," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 28, no. 1, pp. 69–74, Jan. 2006.
- [11] Jayadeva, R. Khemchandani, and S. Chandra, "Twin support vector machines for pattern classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 29, no. 5, pp. 905–910, May 2007.
- [12] M. Arun Kumar and M. Gopal, "Least squares twin support vector machines for pattern classification," *Expert Syst. Appl.*, vol. 36, no. 4, pp. 7535–7543, May 2009.
- [13] Y.-H. Shao, W.-J. Chen, and N.-Y. Deng, "Nonparallel hyperplane support vector machine for binary classification problems," *Inf. Sci.*, vol. 263, pp. 22–35, Apr. 2014.
- [14] M. Tanveer, A. Sharma, and P. N. Suganthan, "General twin support vector machine with pinball loss function," *Inf. Sci.*, vol. 494, pp. 311–327, Aug. 2019.
- [15] L. Liu, M. Chu, R. Gong, and L. Zhang, "An improved nonparallel support vector machine," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 11, pp. 5129–5143, Nov. 2021.
- [16] S. Rezvani and J. Wu, "Handling multi-class problem by intuitionistic fuzzy twin support vector machines based on relative density information," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 12, pp. 14653–14664, Dec. 2023.
- [17] Y.-H. Shao, C.-H. Zhang, X.-B. Wang, and N.-Y. Deng, "Improvements on twin support vector machines," *IEEE Trans. Neural Netw.*, vol. 22, no. 6, pp. 962–968, Jun. 2011.
- [18] Y. Xu, Z. Yang, and X. Pan, "A novel twin support-vector machine with pinball loss," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 2, pp. 359–370, Feb. 2017.
- [19] D. Xie, F. Nie, and Q. Gao, "On the optimal solution to maximum margin projection pursuit," *Multimedia Tools Appl.*, vol. 79, nos. 47–48, pp. 35441–35461, Dec. 2020.
- [20] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015, pp. 1–14.
- [21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [22] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2022, pp. 11976–11986.
- [23] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [24] A. Dosovitskiy et al., "An image is worth 16×16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020, pp. 1–21.
- [25] A. Mao, M. Mohri, and Y. Zhong, "Cross-entropy loss functions: Theoretical analysis and applications," in *Proc. 40th Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 23803–23828.
- [26] J. Wang, K. Feng, and J. Wu, "SVM-based deep stacking networks," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 5273–5280.
- [27] D. Diaz-Vico, J. Prada, A. Omari, and J. Dorronsoro, "Deep support vector neural networks," *Integr. Comput.-Aided Eng.*, vol. 27, no. 4, pp. 389–402, Sep. 2020.
- [28] X. Xie, Y. Li, and S. Sun, "Deep multi-view multiclass twin support vector machines," *Inf. Fusion*, vol. 91, pp. 80–92, Mar. 2023.
- [29] M. Li and Z. Yang, "Deep twin support vector networks," in *Proc. CAAI Int. Conf. Artif. Intell.*, 2022, pp. 94–106.
- [30] M. Kumar, B. Kumar, P. Sharma, R. Sharma, M. Al-Dhaifallah, and A. Shakoor, "Attentive deep learning with randomized vector energy least square twin support vector machine for Alzheimer's disease diagnosis," *Comput. Electr. Eng.*, vol. 126, Aug. 2025, Art. no. 110412.
- [31] J. Platt, "Sequential minimal optimization: A fast algorithm for training support vector machines," *Microsoft*, Redmond, WA, USA, Tech. MSR-TR-98-14, 1998.
- [32] J. Xie, K. Hone, W. Xie, X. Gao, Y. Shi, and X. Liu, "Extending twin support vector machine classifier for multi-category classification problems," *Intell. Data Anal.*, vol. 17, no. 4, pp. 649–664, Jun. 2013.
- [33] Y. Tian, Z. Qi, X. Ju, Y. Shi, and X. Liu, "Nonparallel support vector machines for pattern classification," *IEEE Trans. Cybern.*, vol. 44, no. 7, pp. 1067–1079, Jul. 2014.
- [34] C. Wang, Q. Ye, P. Luo, N. Ye, and L. Fu, "Robust capped L1-norm twin support vector machine," *Neural Netw.*, vol. 114, pp. 47–59, Jun. 2019.
- [35] C. Yuan, L. Yang, and P. Sun, "Correntropy-based metric for robust twin support vector machine," *Inf. Sci.*, vol. 545, pp. 82–101, Feb. 2021.
- [36] J. Zhang, Z. Lai, H. Kong, and L. Shen, "Robust twin bounded support vector classifier with manifold regularization," *IEEE Trans. Cybern.*, vol. 53, no. 8, pp. 5135–5150, Aug. 2023.
- [37] Z.-X. Yang, Y.-H. Shao, and X.-S. Zhang, "Multiple birth support vector machine for multi-class classification," *Neural Comput. Appl.*, vol. 22, no. S1, pp. 153–161, May 2013.
- [38] D. Tomar and S. Agarwal, "A comparison on multi-class classification methods based on least squares twin support vector machine," *Knowl.-Based Syst.*, vol. 81, pp. 131–147, Jun. 2015.

- [39] B. Schölkopf, A. J. Smola, and F. Bach, *Learning With Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. Cambridge, MA, USA: MIT Press, 2002.
- [40] X. He, S. Yan, Y. Hu, P. Niyogi, and H.-J. Zhang, "Face recognition using Laplacianfaces," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 27, no. 3, pp. 328–340, Mar. 2005.
- [41] C. Li, Q. Liu, W. Dong, F. Wei, X. Zhang, and L. Yang, "Max-margin-based discriminative feature learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 12, pp. 2768–2775, Dec. 2016.
- [42] J. Zhang, Z. Lai, H. Kong, and J. Yang, "Learning the optimal discriminant SVM with feature extraction," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 4, pp. 1–15, Apr. 2025.
- [43] F. Wang, J. Cheng, W. Liu, and H. Liu, "Additive margin softmax for face verification," *IEEE Signal Process. Lett.*, vol. 25, no. 7, pp. 926–930, Jul. 2018.
- [44] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive angular margin loss for deep face recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 4685–4694.
- [45] J. Lee, H. Lee, K. Hwang, and N. Kwak, "Deep support vectors," in *Proc. 38th Annu. Conf. Neural Inf. Process. Syst.*, 2024, pp. 118155–118181.
- [46] L. Bai, Z. Wang, Y.-H. Shao, and N.-Y. Deng, "A novel feature selection method for twin support vector machine," *Knowl.-Based Syst.*, vol. 59, pp. 1–8, Mar. 2014.
- [47] S. Gao, Q. Ye, and N. Ye, "1-norm least squares twin support vector machines," *Neurocomputing*, vol. 74, no. 17, pp. 3590–3597, Oct. 2011.
- [48] J. Xie, J. Qiu, D. Zhang, and R. Zhang, "A novel fuzzy twin support vector machine based on centered kernel alignment," *Soft Comput.*, vol. 28, no. 20, pp. 1–19, Oct. 2024.
- [49] A. Quadir, M. Sajid, and M. Tanveer, "Granular ball twin support vector machine," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 7, pp. 1–10, Jul. 2025.
- [50] C. Jimenez-Castaño, A. Álvarez Meza, D. Cárdenas-Peña, A. Orozco-Gutiérrez, and J. Guerrero-Erazo, "Krein twin support vector machines for imbalanced data classification," *Pattern Recognit. Lett.*, vol. 182, pp. 39–45, Jun. 2024.
- [51] K. Crammer and Y. Singer, "On the algorithmic implementation of multiclass kernel-based vector machines," *J. Mach. Learn. Res.*, vol. 2, no. 2, pp. 265–292, 2002.
- [52] O. K. Oyedotun, K. A. Ismaeil, and D. Aouada, "Why is everyone training very deep neural network with skip connections?," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 5961–5975, Sep. 2023.
- [53] O. Sener and V. Koltun, "Multi-task learning as multi-objective optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 527–538.
- [54] S. Boyd, S. P. Boyd, and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [55] J.-A. Désidéri, "Multiple-gradient descent algorithm (MGDA) for multiobjective optimization," *Comp. Rendus. Mathématique*, vol. 350, nos. 5–6, pp. 313–318, Mar. 2012.
- [56] I. Giagkiozis and P. J. Fleming, "Methods for multi-objective optimization: An analysis," *Inf. Sci.*, vol. 293, pp. 338–350, Feb. 2015.
- [57] M. Momma, C. Dong, and J. Liu, "A multi-objective/multi-task learning framework induced by Pareto stationarity," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 15895–15907.
- [58] M. Belkin, P. Niyogi, and V. Sindhwani, "Manifold regularization: A geometric framework for learning from labeled and unlabeled examples," *J. Mach. Learn. Res.*, vol. 7, no. 85, pp. 2399–2434, 2006.
- [59] A. Rudi, L. Carratino, and L. Rosasco, "FALKON: An optimal large scale kernel method," in *Proc. 31st Conf. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 3888–3898.
- [60] F. Nie, R. Zhang, and X. Li, "A generalized power iteration method for solving quadratic problem on the Stiefel manifold," *Sci. China Inf. Sci.*, vol. 60, no. 11, pp. 273–297, Nov. 2017.
- [61] H. Zou, T. Hastie, and R. Tibshirani, "Sparse principal component analysis," *J. Comput. Graph. Statist.*, vol. 15, no. 2, pp. 265–286, 2006.
- [62] Z. Wen and W. Yin, "A feasible method for optimization with orthogonality constraints," *Math. Program.*, vol. 142, nos. 1–2, pp. 397–434, Dec. 2013.
- [63] A. Beck, "The 2-coordinate descent method for solving double-sided simplex constrained minimization problems," *J. Optim. Theory Appl.*, vol. 162, no. 3, pp. 892–919, Sep. 2014.
- [64] T.-K. Kim and R. Cipolla, "Canonical correlation analysis of video volume tensors for action categorization and detection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 8, pp. 1415–1428, Aug. 2009.
- [65] K. Lang, "NewsWeeder: Learning to filter netnews," in *Machine Learning Proceedings 1995*. Amsterdam, The Netherlands: Elsevier, 1995, pp. 331–339.
- [66] J. J. Hull, "A database for handwritten text recognition research," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 16, no. 5, pp. 550–554, May 1994.
- [67] M. Cimpoi, S. Maji, I. Kokkinos, S. Mohamed, and A. Vedaldi, "Describing textures in the wild," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2014, pp. 3606–3613.
- [68] M.-E. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes," in *Proc. 6th Indian Conf. Comput. Vis., Graph. Image Process.*, Dec. 2008, pp. 722–729.
- [69] A. Krizhevsky, *Learning multiple layers of features from tiny images*, Dept. Comput. Sci., Univ. of Toronto, Toronto, ON, Canada, Tech. 2009.
- [70] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [71] Y. Netzer, "Reading digits in natural images with unsupervised feature learning," in *Proc. NeurIPS Workshop Deep Learn. Unsupervised Feature Learn.*, 2011, pp. 1–9.
- [72] O. L. Mangasarian and D. R. Musicant, "Successive overrelaxation for support vector machines," *IEEE Trans. Neural Netw.*, vol. 10, no. 5, pp. 1032–1037, Sep. 1999.
- [73] P.-H. Chen, R.-E. Fan, and C.-J. Lin, "A study on SMO-type decomposition methods for support vector machines," *IEEE Trans. Neural Netw.*, vol. 17, no. 4, pp. 893–908, Jul. 2006.
- [74] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016, pp. 1–16.
- [75] L. Xu, L. Chen, R. Wang, F. Nie, and X. Li, "Joint feature and differentiable k -NN graph learning using Dirichlet energy," in *Proc. 37th Conf. Neural Inf. Process. Syst. (NeurIPS)*, vol. 36, 2023, pp. 24497–24518.
- [76] J. Ke, C. Gong, T. Liu, L. Zhao, J. Yang, and D. Tao, "Laplacian welsch regularization for robust semisupervised learning," *IEEE Trans. Cybern.*, vol. 52, no. 1, pp. 164–177, Jan. 2022.
- [77] J. M. Danskin, "The theory of max-min, with applications," *SIAM J. Appl. Math.*, vol. 14, no. 4, pp. 641–664, 1966.



Junhong Zhang is currently pursuing the Ph.D. degree with Shenzhen University, Shenzhen, China.

He is with the College of Computer Science and Software Engineering, Shenzhen University, and also with Shenzhen Institute of Artificial Intelligence and Robotics for Society, Shenzhen. His research interests include optimization in machine learning, kernel methods, and support vector machines (SVMs).



Zhihui Lai (Member, IEEE) received the B.S. degree in mathematics from South China Normal University, Guangzhou, China, in 2002, the M.S. degree from Jinan University, Guangzhou, in 2007, and the Ph.D. degree in pattern recognition and intelligence system from Nanjing University of Science and Technology (NUST), Nanjing, China, in 2011.

He was a Research Associate, a Post-Doctoral Fellow, and a Research Fellow with The Hong Kong Polytechnic University, Hong Kong. He has published over 200 scientific articles. His research interests include face recognition, image processing and content-based image retrieval, pattern recognition, compressive sensing, human vision modelization, and applications in the fields of intelligent robot research.

Dr. Lai is currently an Associate Editor of *International Journal of Machine Learning and Cybernetics*. For more information including all articles and related codes, the readers are referred to the website (<http://www.scholat.com/laizhihui>).



Jie Zhou (Member, IEEE) received the Ph.D. degree from Tongji University, Shanghai, China, in 2011.

He is currently a Research Fellow with the School of Mathematics and Statistics, Changsha University of Science and Technology, Changsha, China. He was a Research Fellow with the School of Artificial Intelligence, Shenzhen University, Shenzhen, China. From 2010 to 2011, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, AB, Canada. From 2017 to 2018, he was an Associate

Researcher with The Hong Kong Polytechnic University, Hong Kong. His current research interests include uncertainty analysis, pattern recognition, data mining, and intelligent systems.



Guangfei Liang received the Bachelor of Science degree in mathematics and applied mathematics from South China University of Technology, Guangzhou, China, in 2022, and the Master of Science degree in computer science from the Institute of Computer Vision, Shenzhen University, Shenzhen, China, in 2025, under the supervision of Prof. Zhihui Lai.

His research interests focus on graph neural networks (GNNs), large language models (LLMs), and support vector machines (SVMs).